COMPUTER GRAPHICS

Graphics input - output devices: Raster scan Displays - Random scan displays - Direct view storage tubes - Flat panel displays - Mouse - Track Ball - Joy Stick - Digitizers - Touch panels.	(4)
GRAPHICAL USER INTERFACE AND INTERACTIVE INPUT METHODS: The user dialog - Input of graphical data - Input function - Interactive picture construction techniques - Virtual reality environments.	(3)
Two Dimensional Graphics: Basic transformations - Matrix representation and homogeneous coordinates - composite transformations - Line drawing algorithms: DDA and Bresenham's algorithms - Circle generation algorithms: Mid point circle algorithm - Point clipping - Line clipping: Cohen Sutherland algorithm - Polygon clipping: Sutherland Hodgeman algorithm - Line covering.	(10)
Raster Graphics: Fundamentals: generating a raster image, representing a raster image, scan converting a line drawing, displaying characters, speed of scan conversion, natural images - Solid area scan conversion: Scan conversion of polygons, Y-X algorithm, properties of scan conversion algorithms - Interactive raster graphics: painting model, moving parts of an image, feed back images	(10)
Curves and surfaces: Parametric representation of curves - Bezier curves - B-Spline curves - parametric representation of surfaces - Bezier surfaces - curved surfaces - ruled surfaces - quadric surfaces.	(6)
Three Dimensional Graphics: 3D transformations - viewing 3D graphical data - orthographic, oblique, perspective projections - hidden lines and hidden surface removal.	(6)
Animation Graphics: Design of Animation sequences - animation function - raster animation - key frame systems - motion specification -morphing - tweening.	(6)
Computer Graphics realism: Tiling the plane - Recursively defined curves - Koch curves - C curves - Dragons - space filling curves - fractals - Grammar based models - graftals - turtle graphics - ray tracing.	(10)
	(56)

REFERENCES

1. Donald Hearn and Pauline Baker M, " Computer Graphics", Pearson Education, 2002.
2. Rankin John R., "Computer Graphics Software Construction", Prentice Hall., 1989.
3. Foley James D., Vandam Andries and Hughes John F., "Computer Graphics : Principles and Practice", Pearson Education, 2002.
4. William M. Newmann and Robert F. Sproull, "Principles of Interactive Computer Graphics", McGraw Hill, 2002.
5. Hill F.S. Jr., "Computer Graphics", Maxwell Macmillan International editions, 2001.
6. Roy. A. Plastock and Gordon Kalley, "Theory and Problems of Computer Graphics", Schaum's outline series, McGraw Hill International editions, 2000.

Introduction to Computer Graphics

DISPLAY DEVICES

a. Soft Copy
    i. Cathode Ray Tube
        1. Refresh
            a. Raster Scan
            b. Random Scan
        2. Non-Refresh
            a. DVST
    ii. Flat Panel Display
        1. Emissive
            a. Plasma Panel
            b. Thin-film Electro Luminescent Display
            c. LED
        2. Non Emissive
b. Hard Copy
    i. Printer
        1. Impact
            a. Dot Matrix
            b. Line
            c. Daisy Wheel
        2. Non-Impact
            a. InkJet
            b. Laser
    ii. Plotter
        1. Drum
        2. Flatbed.

Display System:

The purpose of the display system may be to display character, text,g raphics and video. It has three parts:

- Display controller
- Monitor
- Cable

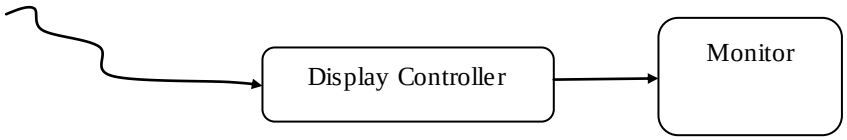


Fig. Display System

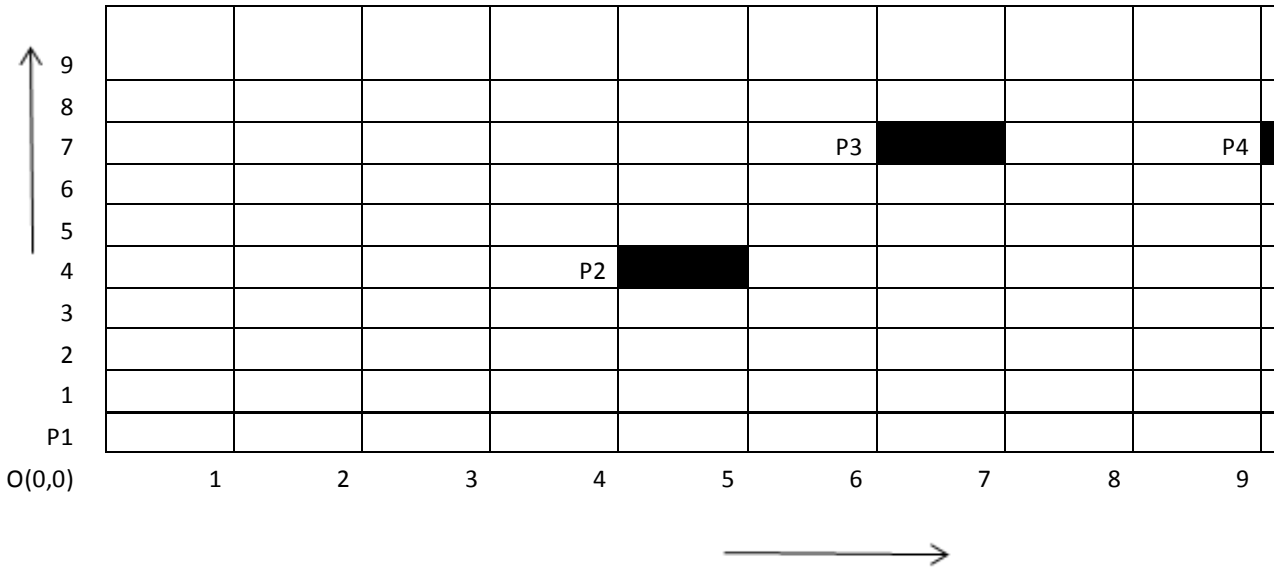
The display controller sits between computer and display devices, receiving information from computers and converting it into signals acceptable to device. Task done by display controllers includes voltage level conversions between computer and display devices, buffering to compensate for differences in speed of operation and generation of lines segments and text characters.

Video Cards
Video cards are physical hardware circuit boards that connect to the motherboards. Video cards are also now being placed onto the computer motherboard to help bring the cost down on computers.
Video cards must specify its video standards, allowing end users to know what video cards may or may not be capable of doing.

Video Card standards
MDA, CGA, EGA, PGA, VGA, XGA, SVGA, SXGA, UXGA, WSXGA, WUXGA, WXGA.

Pixel

The image is displayed as a collection of phosphor dots of regular shapes. These regular shapes are called pixels (pixel elements/pels). These pixels could be rectangles, circles, squares.
A pixel is the smallest addressable portion of an image or display that a computer is capable of printing or displaying.
The origin of the referenced coordinate system is located in the lower left corner of the display screen. P3



P1(0,0), P2(4,4), P3(6,7), P4(9,7)

Any image that is displayed on the monitor is made of thousands of such mall pixels. Each pixel has a particular color and intensity value. It is a measure of screen resolution.

Dead Pixels

A dead pixel is not a common issue for monitors, however this issue can still occur. When this does occur, it is common on many monitors that an entire row or entire column of pixels to go out. Most monitor manufacturers do not have a policy or warranty for this issue and when this occurs then replace the monitor if in warranty.

Dot Pitch

The internal surface of monitor screen is coated with red, green and blue phosphor material that glows when struck by a stream of electrons. This coated material is arranged into an array of million of tiny cells-red, green and blue usually called dots.

Dot pitch is the measurement of the diagonal distance between two liked colored (red, green or blue) pixels on a display screen. It is measured in mm. Usually monitors are available with dot pitch specification 0.25 mm to 0.40 mm. The smaller the dot pitch, the higher the resolution, sharpness and detail of the image displayed.

Resolution

Image Resolution: It refers to pixel spacing. (i.e) the distance from one pixel to the next pixel.

Screen Resolution: It refers to number of pixels in the horizontal and vertical directions on the screen.

Bit Depth

It refers to the number of bits assigned to each pixel in the image and number of colors that can be created from those bits. It specifies number of colos that a monitor can display. It is also referred to as color depth.

Video Standards	Resolution	Bit Depth	Storage required	Number of Colors
Monochrome		1		2
CGA (Color Graphics Adapter)	320 x 200	2	16000 Byte	$2^2 = 4$
EGA (Enhanced Graphics Adapter)	640 x 350	4		$2^4 = 16$
VGA (Video Graphics Array)	640 x 480	8		$2^8 = 256$
XGA(Extended Graphics Array)	800 x 600	16		$2^{16} = 65,536$
SVGA(Super Video Graphics Array)	1280 x 1024	24		$2^{24} = 16,777,216$

Aspect Ratio

It is the ratio of the horizontal points to the vertical points necessary to produce equal-length lines in both directions on the screen.

$$\text{Aspect Ratio} = \frac{No.of\ horizontal\ points}{No.of\ vertial\ Points}$$

Most computers have landscape monitors and most software is designed for that mode. Landscape monitors have aspect ration more than 1, usually in the ratio 4:3.

Refresh Rate

It is the number of times per second the pixels are recharged so that image does not flicker.

It is measured in Hertz(Hz), which is also called as frame rate, Normally refresh rate varies from 60 to 75;. A refresh rate of 60 Hz means image is redrawn 60 times a second.

The higher the refresh rate, the lesser is the flickering.

Flicker

A flicker issue (unsteadiness in an image) can occur when the video refresh rate is too low, other video related issues. When this type of issue occurs, users feel more eyestrain.

Raster Refresh Graphics Displays

In Storage tube and calligraphic line drawing refresh display, a straight line can be drawn directly from any addressable point to another address.

Raster graphics device is a matrix of discrete cells, each of which can be made bright.

In Raster device, a straight line cannot be drawn straight. It can only be approximated by a series of dots close to the path of the line.

Only in the special cases of completely horizontal, vertical or for square pixels 45^o, a straight line of dots result. All other lines appear as a series of stair steps; this is called "aliasing" or "jaggies".

Frame Buffers

- It is a large contiguous piece of computer memory. At a minimum there is one memory bit for each pixel (picture element / pels) in the raster. This amount of memeory is called a bit plane.
- 1024 x 1024 element square raster requires
  $2^{10} \times 2^{10} = 2^{20}$ bits in a single bit plane (Black & White).
- Frame buffer is a digital device.
- Raster CRT is an analog device.
- Conversion from digital representation to an anolog signal takes place when information is read from the frame buffer and displayed on the raster CRT. This process is done by DAC (Digital to Analog Converter).

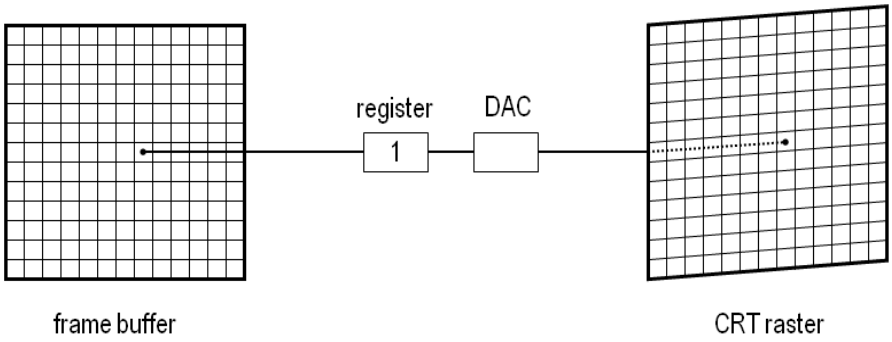


Fig. Single Bit Plane Black and White Frame Buffer

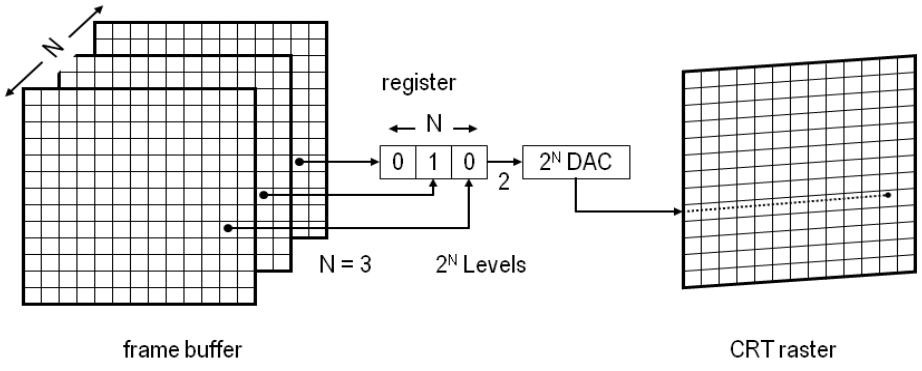


Fig. Frame buffer for N-bit nanochrome display

Color Or Gray Levels

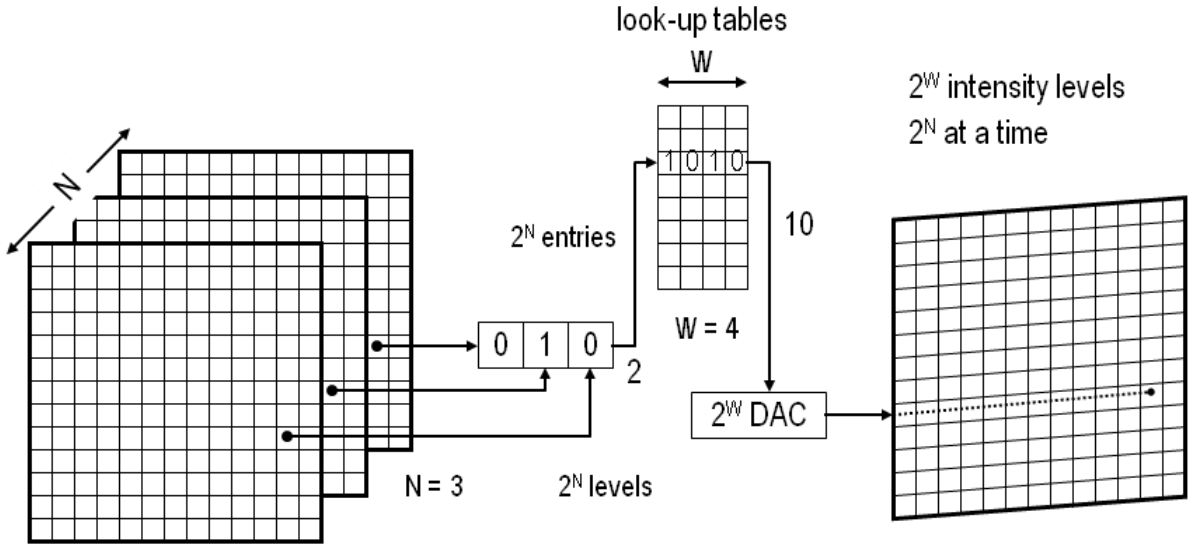
- The intensity of each pixel is controlled by a corresponding pixel location in each of the N bit planes.
- The binary value (0/1) from each of the N bit planes is loaded into corresponding positions in a register.
- The resulting binary number is interpreted as in intensity level between 0 (dark) and 2^N -1 (full intensity).
- This is converted to an analog voltage between 0 and maximum voltage if the electron gun by the DAC.

  2^N intensity (totally) available.

  (e.g) 3 bit plane frame buffer for a 1024 x 1024 raster requires 3 x 1024 x 1024 memory bits.

Increasing the intensity levels (Lookup Table)

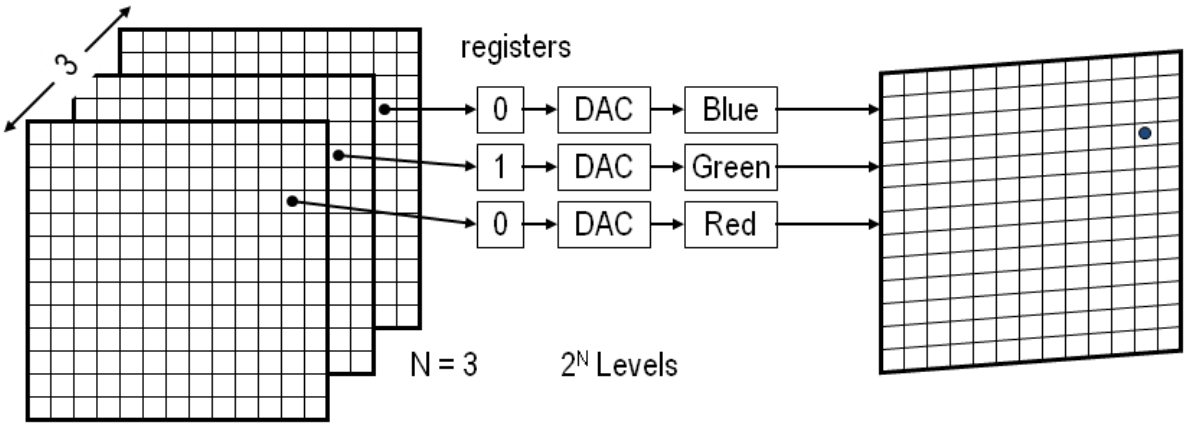
- Upon reading the bit planes in the frame buffer, the resulting number is used as index into the lookup table. The lookup table must contain 2^N entities.
- Each entry in the lookup table is W bits wide. W can be greater than N. but only 2^N entries (intensities) available.



Color CRT frame buffer CRT raster

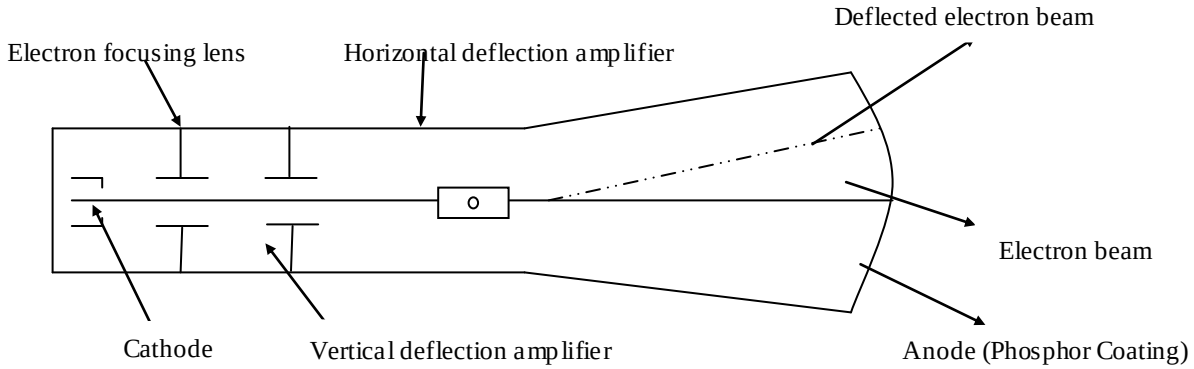
Frame buffer for color CRT

	Red	Green	Blue
Black	0	0	0
Red	1	0	0
Green	0	1	0
Blue	0	0	1
Yellow	1	1	1
Cyan	0	1	1
Magenta	1	0	1
White	1	1	1

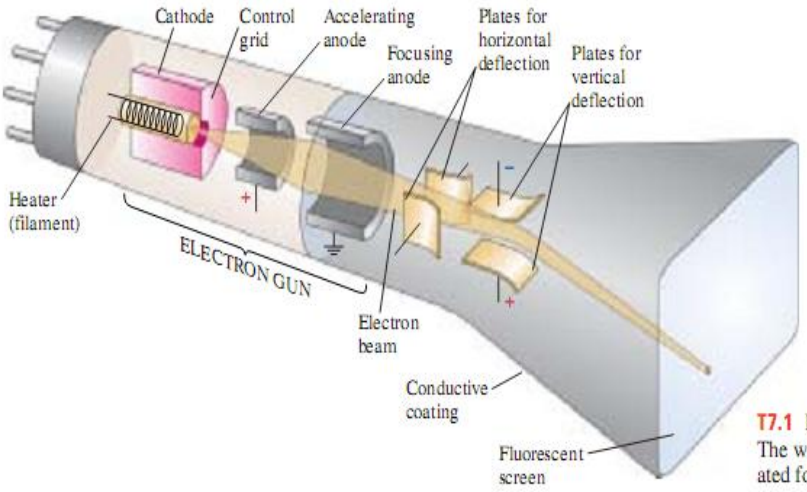


CATHODE RAY TUBE BASICS

- The CRT used in video monitors is shown. (link : http://www.tpub.com/neets/book6/21e.htm for more details).



- A cathode is heated until electrons "boil" off in a diverging cloud (electrons repel each other because they have the same charge).



T7.1 Basic elements of a cathode-ray tube. The width of the electron beam is exaggerated for clarity.

Components of CRT

- Electron Gun
- Control Electrode
- Focusing Electrode
- Deflection Yoke (Electrostatic deflection plate / Magnetic deflection coil)
- Phosphor coated screen.

Electron Gun: Electron gun contains two basic parts: a heater and a cathode. Electrons are generated by a cathode heated by an electric filament, surrounding the cathode is a control grid, with a hole at one end to allow electrons to escape. The control grid is kept at a lower potential than cathode. These electrons (negatively charged) are accelerated towards the CRT screen by a high positive voltage applied near the screen or by an accelerating anode.

Control Electrode: it is used to regulate the flow of electrons. The control electrode is connected to an amplifier which, in turn, is connectd to the output circuitry of the computer, thus allowing the computers to control the electron beam in turned off and on. We can modify the rate of flow of electrons, or beam current and control brightness of image.

Focusing Electron: this system helps to converge the cloud of electrons to a small spot as it touches the CRT screen. It is used to create a clear picture by focusing the electrons into a narrow beam, in other words, it focuses the electron beam to converge into a small spit as it strikes the phosphors. Otherwise the electrons would repel each other, and beam would spread out as it approaches the screen. Focusing is accomplished wih either electric or magnetic fields. The effect of the focusing electrode on the on the electron beam resembles that of a flass lens on the light waves.

Electrostatic focusing is commonly used in television and computer graphics monitors. With electrostatic focusing, the electron beam passes through a positiviely charged metal cylinder that forms an electrostatic lens. The action of the electrostatic lens focuses the electron beam at the center of the screen, in exactly the same way that an optical lens focuses a beam of light at a particular focal distance.

Deflection Yoke:

It is used to control the direction of the electron beam. The deflection yoke creates an electric or magnetic field which will bend the electron beam as it passes through the field.

When electrostatic deflection is used, two pairs of parallel plates are mounted inside the CRT envelope. One pair of plates is horizontal deflection to control the vertical deflection and other pair is mounted vertical to control horizontal deflection. In an alternative way for deflection system CRT are constructed with magnetic deflection coils, these coils are mounted on the outside of the CRT envelope. Two pairs of coils are used. One pair is mounted on the top and bottom of the neck and other pair is mounted on opposite side to the neck. Horizontal deflection is accomplished with one pair of coils and vertical deflection by the other pair. In a conventional CRT the yoke is connected to a sweep or scan generator. The scan generator sends out an oscillatory sawtooth current that, in turn causes the deflection yoke to apply a varying magnetic field to the electron beam's path. The oscillatory voltage potential causes the electron beam to move across the CRT's screen in a regular pattern.

Phosphorous-coated screen:

Inner surface of CRT is coated with special crystals called phosphors, which have a unique property that allows the entire system to work. Phosphors glow when they are attacked by a high-energy electron beam. They continue to glow for a distinct period of time after being exposed to electron beam. The glow given off by the phosphor during exposure to the electron is known as flouorescenec, the continuing glow given off after the beam is removed is known as phosphorescence and the duration of phosphorescence is known as the phosphors persistence.

Lower persistence phosphors require higher refresh rate to maintain a picture on the screen without flicker. Higher persistence phosphors require lower refresh rate to maintain a picture on screen without flicker. A phosphor with low persistence is useful for animation. A phosphor with high persistence is useful for display highly complex.

Working of CRT: A CRT is an evacuated glass tube, with a heating element on one end and a phosphor coated screen on the other end. When a current flows through this heating element, called a filament, the electrons actually boil off the filament. These free electrons are attracted to a strong positive charge from the outer surface of the focusing anode cylinder (sometimes called an electrostatic lens). However, the iside of the cylinder has a weaker negative charge. Thus, when the electrons head towards the anode they are forced into a beam and accelerated by the repulsion of the inner cylinder walls in just the way that water speeds up when it flows through a smaller diameter pipe. By the time the electrons get out they are going so fast that they fly past the cathode they were heading for.

The next thing that the electrons run into are two sets of weakly charged deflection plates. These plates have opposite charges, one positive and the other negative. While their charge is not strong enough to capture the fast moving electrons they do influence the path of the beam. The first set displaces the beam up and down, and the second displaces the beam left and right. The electrons are sent flying out of the neck of the bottle, until they smash into the phosphor coating on the other end of the bottle. The impact of this collision o the outvalence bands of the phosphor compounds knowcks some of the electrons to jump into another band. This causes a few photons to be generated, and results in our seeing a spot on the CRT's face.

Phosphor differ in following.

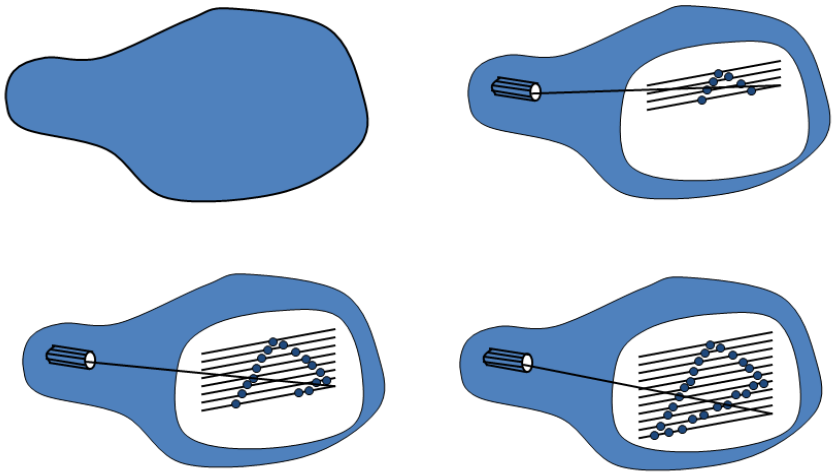
- Color : Different phosphors have different colors.
- How long they continue to emit light (the excited electrons returning to the ground state) after the CRT beam is removed

Persistence -- It is defined as the time it takes the emitted light from the screen to decay to one-tenth of its original intensity.

Interlacing :

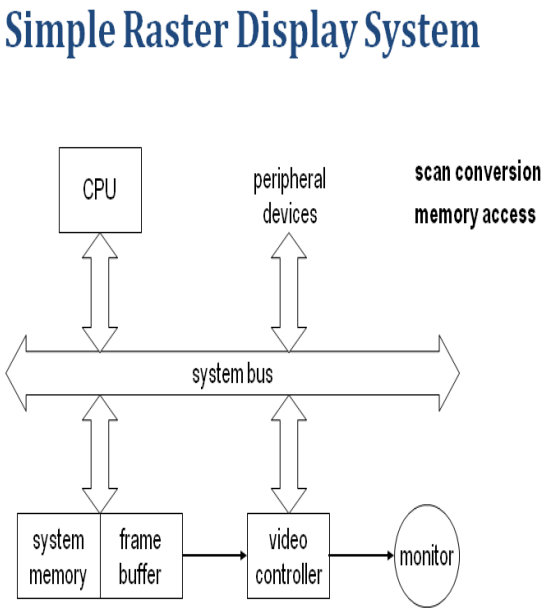
On some raster scan systems each frame is displayed in two passes using an interlaced refresh procedure. In the first pass, the beam sweeps (odd scan lines) across every other scan line from top to bottom. Then after the vertical re-trace, the beam sweeps out the remaining (even scan lines). Interlacing of the scan lines in this way allows us to see the entire screen displayed in one-half the time it would have taken to sweep across all the scan lines at once from top to bottom. Interlacing is primarily used with slower refreshing rates.

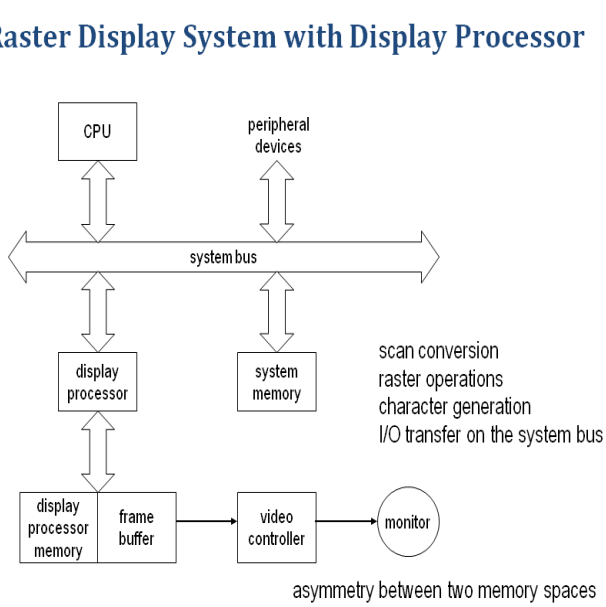
Raster Scan Display



A raster-scan system displays an object as a set of points across each screen scan line

Architecture of Raster Scan Display





Raster-scan Systems :- Interactive raster graphics systems typically employ several processing units. In addition to the central processing unit or CPU, a special-purpose processor, called the video controller or display controller, is used to control the operation of the display device. Organization of a simple raster system. Here, the frame buffer can be anywhere in the system memory, and the video controller accesses the frame buffer to refresh the screen. In addition to the video controller, more sophisticated raster systems employ other processors as coprocessors.

A fixed area of the system memory is reserved from the frame buffer, and the video controller is given direct access to the frame-buffer memory. Frame-buffer locations and the corresponding screen positions are referenced in cartesian coordinates. For many graphics monitors, the coordinate origin is defined at the lower left screen corner. The screen surface is then represented as the first quadrant of a two-dimensional system, with positive values increasing to the right and positive values from bottom to top. The basic refresh operations of the video controller. Two registers are used to store the coordinates of the screen pixels. Initially, the resister is set to 0. The value stored in the frame buffer for this pixel position is then retrieved and used to set the intensity of the CRT beam. Then the resisters is incremented by first, and the process repeated for the next pixel on the top scan line. Since the screen must be refreshed at the rate of 60 frames per second, the simple procedure illustrated can not be accommodate by typical RAM chips. The cycle time is too slow. To speed up pixel processing, video controllers can petrive multiple pixel values from the refresh buffer on each pass. The multiple pixel intensitites are then stored in a separate resister and used to control the CRT beam intensity for a group of adjacent pixels. When that group of pixels has been processed, the next block of pixel values is retrieved from the frame buffer. A number of other operations can be performed by the video controller, besides the basic refreshing operations. For various applications, the video controller can retrieve pixel intensities from different memory areas on different refresh cycles. In high-quality systems, for example, two frame buffers are often provided so that one buffer can be used for refreshing while the other is being filled with intensity values. Then two buffers can switch roles. This provides a fast mechanism for generating, real-time animations, since different views of

moving objects can be successively loaded into the refresh buffers.

Also, some transformations can be accomplished by the video controller. Areas of the screen can be enlarged, reduced, or moved from one location to another during the refresh cycles. In addition, the video controller often contains lookup tables, pm that pixel values in the frame buffer are used to access the lookup table instead of controlling the CRT beam intensity directly. Finally, some systems are designed to allow the video controller to mix the frame-buffer image with an input image from a television camera or other input device.

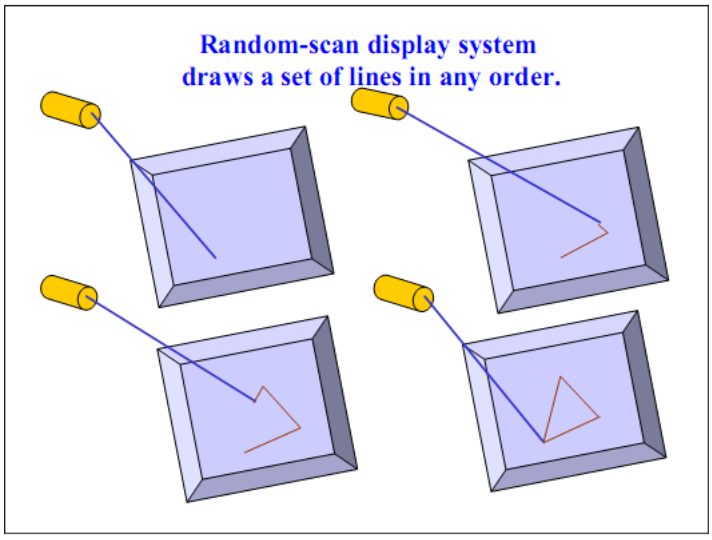
Raster-scan Display Processor :- One way to set up the orangyation of a raster system containing a separate display processor,sometimes referred to as a graphics controller or a display coprocessor.the purpose of the displays processor is to free the CPU from the graphics chores. In addition to the system memory, a separate display-processor memory area can also be provided.

A major task of the display processor is digitizing a picture definition given in an application program into a set of pixel-intensity values for storage in the frame buffer. This digitization process is called scan conversion.
Display processors are also designed to perform a number of additional operations. These functions include generating various line styles, displaying color areas, and performing certain transformations and manipulations on displayed objects. Also, display processors are typically designed to interface with interactive input devices, such as a mouse.

Random-scan Systems

The organization of a simple random-scan system (sometimes called vector scan system). An application program is input and stored in the system memory along with a graphics package. Graphics commands in the application program are translated by the graphics package into a display file stored in the system memory. This display file is then accessed by the display processor to refresh the screen. The display processor cycles.

Calligraphic or Random Scan display system



- Also called Vector, Stroke, Line drawing displays.
- Characters are also made of sequences of strokes (or short lines).
- Vectored – electron beam is deflected from end-point to end-point.
- Random scan - Order of deflection is dictated by the arbitrary order of the display commands.
- Phosphor has short persistence – decays in 10-100 ms.
- The display must be refreshed at regular intervals – minimum of 30 Hz (fps) for flicker-free display.
- Refresh Buffer – memory space allocated to store the display list or display program for the display processor to draw the picture.
- The display processor interprets the commands in the refresh buffer for plotting.
- The display processor must cycle through the display list to refresh the phosphor.
- The display program has commands for point- , line–, and character plotting.
- The display processor sends digital and point coordinate values to a vector generator.
- The vector generator converts the digital coordinate values to analog voltages for the beam-deflection Circuits.
    - The beam-deflection circuits displace the electron beam for writing on the CRT's phosphor coating.

- Recommended refresh rate is 40 – 50 Hz.

❖ Scope of animation with segmentation – mixture of static and dynamic parts of a picture.

When operated as a random-scan display unit, a CRT has the electron beam directed only to the parts of the screen where a picture is to be drawn. Random-scan monitors draw a picture one line at a time and for this reason are also referred to as vector displays (or stroke-writing or calligraphic displays). The component lines of a picture can be drawn and refreshed by a random-scan system in any specified order (fig). A pen plotter operates in a similar way and is an example of a random-scan, hard-copy device.

Refresh rate on a random-scan system depends on the number of lines to be displayed. Picture definition is now stored as a set of line-drawing commands in an area of memory referred to as the refresh display file. Sometimes the refresh display file is called the display list, display program, or simply the refresh buffer. To display a specified picture, the system cycles through the set of commands in the display file, drawing each component line in turn. After all line-drawing commands have been processed, the system cycles back to the first line command in the list. Random-scan displays are designed to draw all the component lines of a picture 30 to 60 times each second. High-quality vector systems are capable of handling approximately 100,000 "short" lines at this refresh rate. When a small set of lines is to be displayed, each refresh cycle is delayed to avoid refresh rates greater than 60 frames per second. Otherwise, faster refreshing of the set of lines could burn out the phosphor. [TOP]

Random-scan systems are designed for line-drawing applications and can-not display realistic shaded scenes. Since picture definition is stored as a Set of line-drawing instructions and not as a set of intensity values for all screen points, vector displays generally have higher resolution than raster systems. Also, vector displays produce smooth line drawings because the CRT beam directly follows the line path. A raster system, in contrast, produces jagged lines that are plotted as discrete point sets.

COLOR CRT

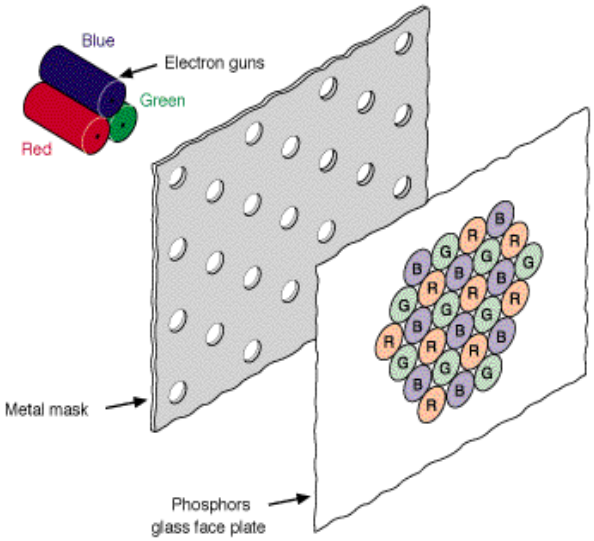
- CRT monitor displays color pictures by using a combination of phosphors that emit different colored light.
- By combining the emitted light from the different phosphors , a range of colors can be generated.
- Color CRTs have
  - 3 phosphor color dots at each pixel position for red , green and blue color
  - Three electron guns one for each color dot
  - A metal *shadow mask* to differentiate the beams

- The 2 basic techniques for producing color CRT displays are

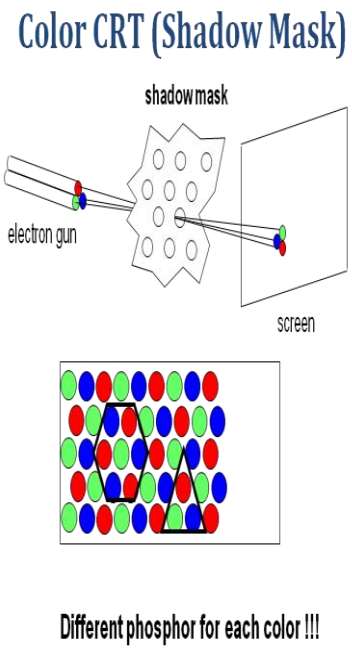
  1. Beam penetration method
  2. Shadow mask method

- Commonly used in raster scan systems because they produce wide range of colours than beam penetration method.

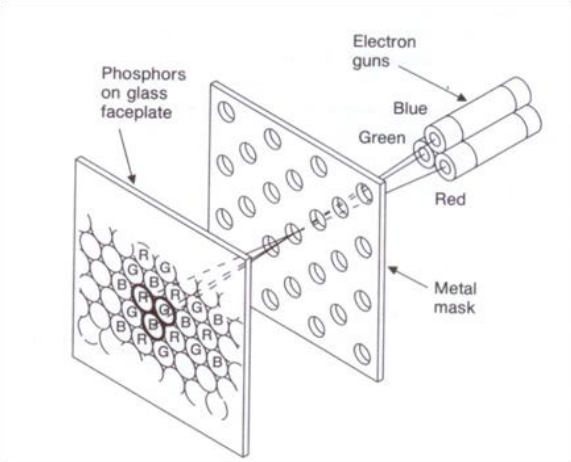
Beam Penetration method



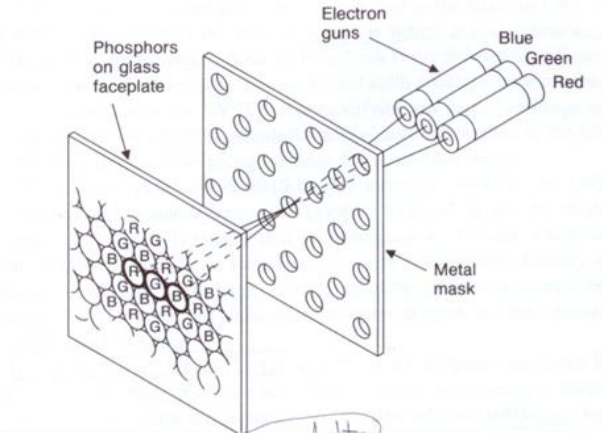
- Commonly used in raster scan systems
  - Two layers of phosphor, usually red and green, are coated onto the inside of CRT screen, and the displayed color depends on how far the electron beam penetrates into the phosphor layers.
  - A beam of slow electrons excites only the outer red layer.
  - A beam of very fast electrons penetrates through the red layer and excites the inner green layer. At intermediate beam speeds, combinations of other colors are produced.



The **shadow mask** is one of two major technologies used to manufacture cathode ray tube (CRT) televisions and computer displays that produce color images (the other is aperture grille and its improved variant Cromaclear). Tiny holes in a metal plate separate the colored phosphors in the layer behind the front glass of the screen. The holes are placed in a manner ensuring that electrons from each of the tube's three cathode guns reach only the appropriately-colored phosphors on the display. All three beams pass through the same holes in the mask, but the angle of approach is different for each gun. The spacing of the holes, the spacing of the phosphors, and the placement of the guns is arranged so that for example the blue gun only has an unobstructed path to blue phosphors. The red, green, and blue phosphors for each pixel are generally arranged in a triangular shape (sometimes called a "triad"). All early color televisions and the majority of CRT computer monitors, past and present, use shadow mask technology. This principle was first proposed by Werner Flechsig in a German patent in 1938.



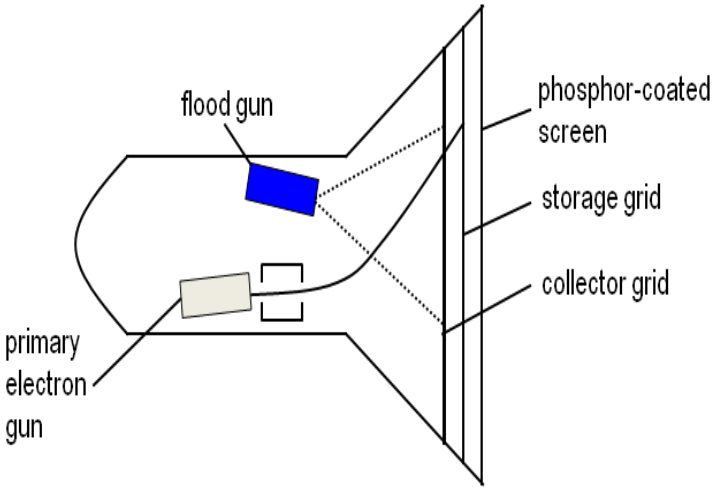
delta-delta shadow-mask CRT



precision in-line delta CRT

Non-Refresh type CRT

Direct View Storage Tubes



• Storage Tube – it is a CRT with a long persistence phosphor.
• Provides flicker-free display
• No refreshing necessary.
• A slow moving electron beam draws a line on the screen.
• Screen has a storage mesh in which the phosphor is embedded.
• Image is stored as a distribution of charges on the inside surface of the screen.
• Limited interactive support.
• Erasing takes about 0.5 seconds. All lines and characters must be erased.
• Slow process of drawing – typically a few seconds are necessary for a complex picture.
• No animation possible with DVST.
• Modifying any part of the image requires redrawing the entire modified image.
• Change in the image requires to generate a new charge distribution in the DVST

Flat Panel Displays

Flat CRT

- A flat CRT is obtained by initially projecting the electron beam parallel to the screen and then reflecting it throught 90^0.
- Reflecting the electron beam significantly reduces the depth of the CRT bottle and, consequently, of the display.
-

Types of Flat panel displays:

I. Plasma Panels.
II. Thin-film electro luminescent display
III. Light-emitted diode

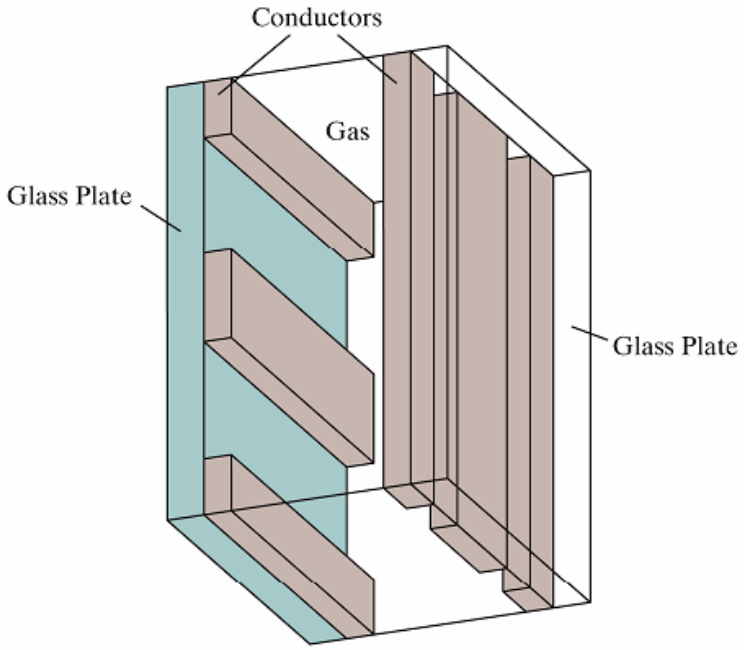


Figure 2-11
Basic design of a plasma-panel display device.

Plasma Panels

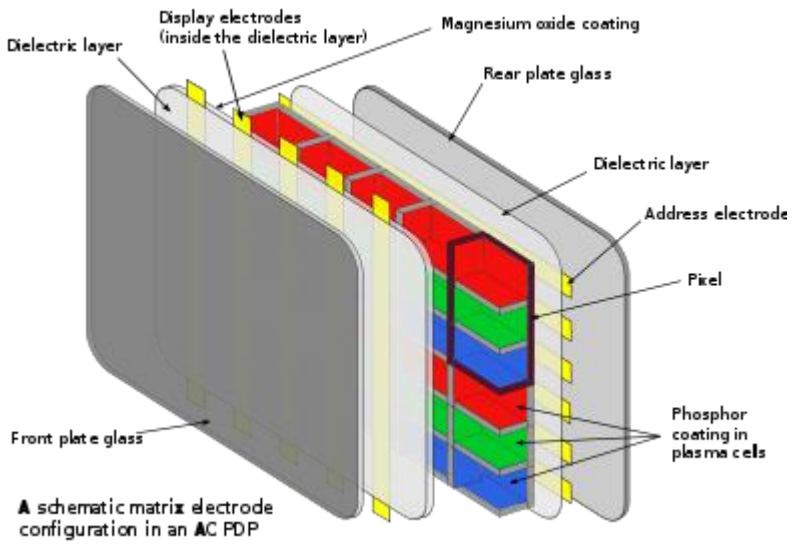
❖ Constructed by filling the region between two glass plates with a mixture of gases that usually includes neon.

❖ A series of vertical conducting ribbons is placed on one glass panel, and a set of horizontal conducting ribbons is built into the other glass panel.

❖ Firing voltages applied to an intersecting pair of horizontal and vertical conductors cause the gas at the intersection of the two conductors to break down into a glowing plasma of electrons and ions.

- Picture definition is stored in a refresh buffer, and the firing voltages are applied to refresh the pixel positions (at the intersection of the conductors) 60 times per second.

Thin-film electroluminescent display



The xenon, neon, and helium gas in a plasma television is contained in hundreds of thousands of tiny cells positioned between two plates of glass. Long electrodes are also put together between the glass plates, in front of and behind the cells. The address electrodes sit behind the cells, along the rear glass plate. The transparent display electrodes, which are surrounded by an insulating dielectric material and covered by a magnesium oxide protective layer, are mounted in front of the cell, along the front glass plate. Control circuitry charges the electrodes that cross paths at a cell, creating a voltage difference between front and back and causing the gas to ionize and form a plasma. As the gas ions rush to the electrodes and collide, photons are emitted.

In a monochrome plasma panel, the ionizing state can be maintained by applying a low-level voltage between all the horizontal and vertical electrodes–even after the ionizing voltage is removed. To erase a cell all voltage is removed from a pair of electrodes. This type of panel has inherent memory and does not use phosphors. A small amount of nitrogen is added to the neon to increase hysteresis.

In color panels, the back of each cell is coated with a phosphor. The ultraviolet photons emitted by the plasma excite these phosphors to give off colored light. The operation of each cell is thus comparable to that of a fluorescent lamp.

Every pixel is made up of three separate subpixel cells, each with different colored phosphors. One subpixel has a red light phosphor, one subpixel has a green light phosphor and one subpixel has a blue light phosphor. These colors blend together to create the overall color of the pixel, the same as a triad of a shadow mask CRT or color LCD. Plasma panels use pulse-width modulation to control brightness: by varying the

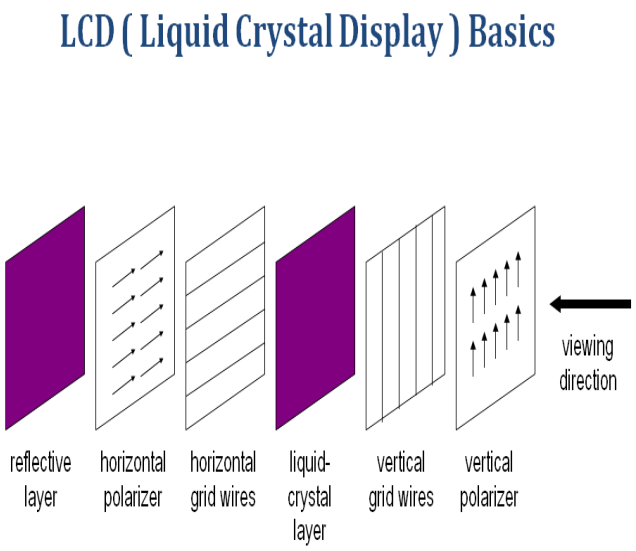
pulses of current flowing through the different cells thousands of times per second, the control system can increase or decrease the intensity of each subpixel color to create billions of different combinations of red, green and blue. In this way, the control system can produce most of the visible colors. Plasma displays use the same phosphors as CRTs, which accounts for the extremely accurate color reproduction when viewing television or computer video images (which use an RGB color system designed for CRT display technology).

Difference between PPD and CRT

1. PPD is less bulky than CRT but cost of construction is very high.
2. Wiring is complex in PPD than CRT.
3. PPD provides excellent brightness than CRT.
4. PPD is used for comparatively large display than CRT>
5. PPD has poor resolution relative to CRT.

Thin-Film Electroluminescent

These are similar in construction to a plasma panel. The only difference is that the enfilement of the region between the glass plates is with a phosphor, such as zinc sulphide doped with manganese, instead of a gas.



LCD is made up of 6 layers

- **vertical polarizer plane;**
- **layer of thin grid wires;**
- **layer of LCDs;**
- **layer of horizontal grid wires;**
- **horizontal polarizer;**

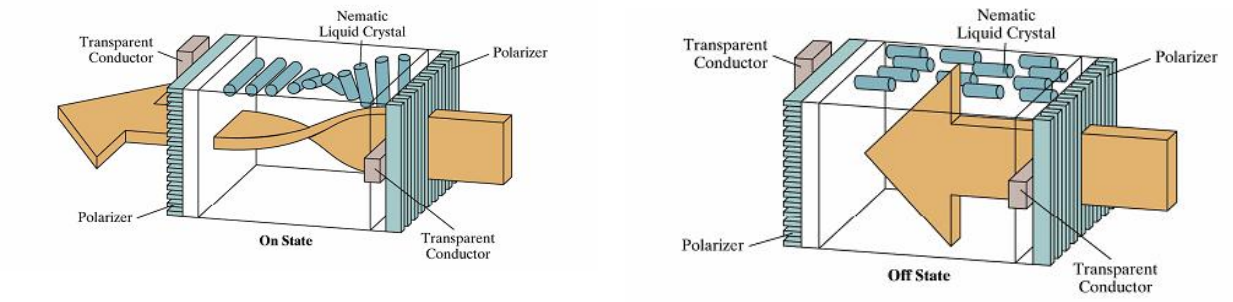
and finally a

- **reflector.**

• Light Emitting Diode (LED)

• A matrix of diodes is arranged to form the pixel positions in the display, and picture definition is stored in a refresh buffer.

• Information is read from the refresh buffer and converted to voltage levels that are
applied to the diodes to produce the light patterns in the display.

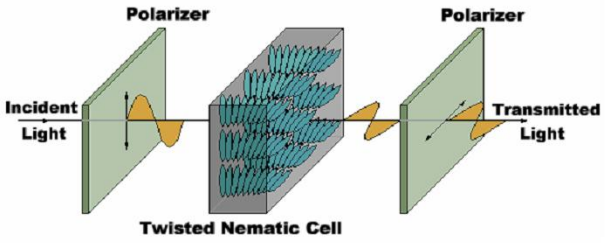


The light-twisting, shutter effect used in the design of most liquid-crystal display devices.

Used in small systems such as laptops and calculators. Produce a picture by passing polarized light from the surroundings or from an internal light source through a liquid-crystal material that can be aligned to either block or transmit the light.

Liquid Crystal Displays (LCDs)

Liquid-crystal: these compounds have a crystalline arrangement of molecules, they flow like a liquid. FPDs use nematic (threadlike) Liquid-crystal compounds that keep the long axes of the rod-shaped molecules aligned.



Passive-matrix LCD

• Two glass plates, each containing a light polarizer that is aligned at a right angle to the other plate, sandwich the liquid-crystal material.

• Rows of horizontal, transparent conductors are built into one glass plate, and columns of vertical conductors are put into the other plate.

• The intersection of the two defines a pixel position. Polarized light passing through the
material is twisted so that it will pass through the opposite polarizer. The light is then reflected back to the viewer.

• To turn off the pixel, we apply a voltage to the two intersecting conductors to align the molecules so that the light is not twisted.

Active-matrix LCD

• This type of LCD is constructing by placing a transistor at each pixel location, using thin-film transistor technology.
• The transistors are used to control the voltage at pixel locations and to prevent charge from gradually leaking out of the liquid-crystal cells.

Other Input Output Devices:

Keyboard

The computer **keyboard** is used to enter text information into the computer, as when you type the contents of a report. The keyboard can also be used to type commands directing the computer to perform certain actions. Commands are typically chosen from an on-screen menu using a mouse, but there are often keyboard shortcuts for giving these same commands.

In addition to the keys of the main keyboard (used for typing text), keyboards usually also have a numeric keypad (for entering numerical data efficiently), a bank of editing keys (used in text editing operations), and a row of function keys along the top (to easily invoke certain program functions). Laptop computers, which don't have room for large keyboards, often include a "fn" key so that other keys can perform double duty (such as having a numeric keypad function embedded within the main keyboard keys).

Improper use or positioning of a keyboard can lead to repetitive-stress injuries.
Some **ergonomic** keyboards are designed with angled arrangements of keys and with built-in wrist rests that can minimize your risk of RSIs.

Most keyboards attach to the PC via a PS/2 connector or USB port (newer). Older Macintosh computers used an ABD connector, but for several years now all Mac keyboards have connected using USB.

Pointing Devices

The graphical user interfaces (GUIs) in use today require some kind of device for positioning the on-screen cursor. Typical pointing devices are: mouse, trackball, touch pad, trackpoint, graphics tablet, joystick, and touch screen.

Pointing devices, such as a mouse, connected to the PC via a serial ports (old), PS/2 mouse port (newer), or USB port (newest). Older Macs used ADB to connect their mice, but all recent Macs use USB (usually to a USB port right on the USB keyboard).

Mouse

The **mouse** pointing device sits on your work surface and is moved with your hand. In older mice, a ball in the bottom of the mouse rolls on the surface as you move the mouse, and internal rollers sense the ball movement and transmit the information to the computer via the cord of the mouse.

The newer **optical mouse** does not use a rolling ball, but instead uses a light and a small optical sensor to detect the motion of the mouse by tracking a tiny image of the desk surface. Optical mice avoid the problem of a dirty mouse ball, which causes regular mice to roll unsmoothly if the mouse ball and internal rollers are not cleaned frequently.

A **cordless** or **wireless mouse** communicates with the computer via radio waves (often using ***BlueTooth*** hardware and protocol) so that a cord is not needed (but such mice need internal batteries).

A mouse also includes one or more buttons (and possibly a scroll wheel) to allow users to interact with the

GUI. The traditional PC mouse has two buttons, while the traditional Macintosh mouse has one button. On either type of computer you can also use mice with three or more buttons and a small scroll wheel (which can also usually be clicked like a button).

Touch pad

Most laptop computers today have a **touch pad** pointing device. You move the on-screen cursor by sliding your finger along the surface of the touch pad. The buttons are located below the pad, but most touch pads allow you to perform "mouse clicks" by tapping on the pad itself.

Touch pads have the advantage over mice that they take up much less room to use. They have the advantage over trackballs (which were used on early laptops) that there are no moving parts to get dirty and result in jumpy cursor control.

Trackpoint

Some sub-notebook computers (such as the IBM ThinkPad), which lack room for even a touch pad, incorporate a**trackpoint**, a small rubber projection embedded between the keys of the keyboard. The trackpoint acts like a little joystick that can be used to control the position of the on-screen cursor.

Trackball

The **trackball** is sort of like an upside-down mouse, with the ball located on top. You use your fingers to roll the trackball, and internal rollers (similar to what's inside a mouse) sense the motion which is transmitted to the computer. Trackballs have the advantage over mice in that the body of the trackball remains stationary on your desk, so you don't need as much room to use the trackball. Early laptop computers often used trackballs (before superior touch pads came along).

Trackballs have traditionally had the same problem as mice: dirty rollers can make their cursor control jumpy and unsmooth. But there are modern optical trackballs that don't have this problem because their designs eliminate the rollers.

Joysticks

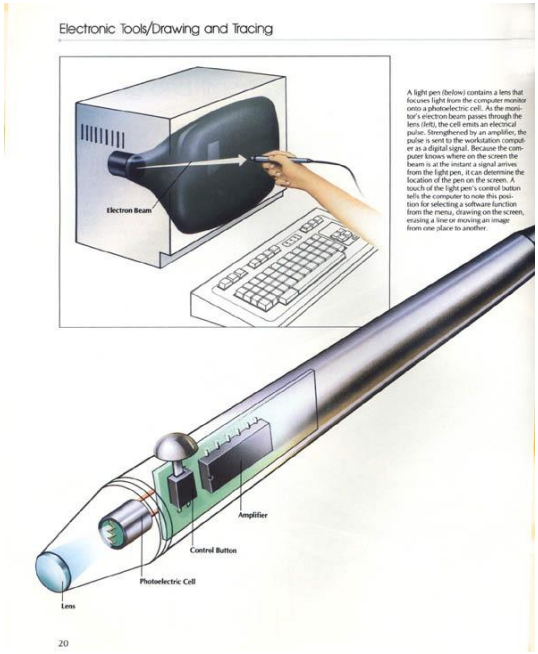
Joysticks and other game controllers can also be connected to a computer as pointing devices. They are generally used for playing games, and not for controlling the on-screen cursor in productivity software. Joystick is a device that moves in all directions and controls the movement of the cursor. The jjoystick offers three types of control: digital, glide and direct.

Digital control allows movement in a limited number of directions such as up, down, left and right.

Glide and direct control allow movements in all directions. Direct control joysticks have the added ability to respond to the distance and speed. A joystick is generally used to control the velocity of the screen cursor movement rather than its absolute position. Joysticks are mainly used for computer games, for other applications, which include controlling machines such as elevators, Cranes, trucks and powered wheelchairsl and flight simulators, training simulators, CAD/CAM systems and for controlling industrial robots.

Joystick elements: 1. Stick 2. Base 3. Trigger 4. Extra buttons 5. Autofire switch 6. Throttle 7. Hat Switch (POV Hat) 8. Suction Cup

Light Pen



It is a pen-like device, which is connected to the machine by a cable. A light pen is a hand-held electro-optical pointing device which when touched to or aimed closely at a connected computer monitor, will allow the computer to determine where on that screen he pen is aimed. It actually does not emit light; its light sensitive –diode would sense the light coming from the screen.

They are sensitive to the short burst of light emitted from the phosphor coating at the instant the electron beam strikes a particular point. Other light sources, such as the background light in the room, are usually not detected by a light pen.

An activated light pen, pointed at a spot on the screen as the electron beam lights up that spot, causes the photocell to respond by generating an electrical pulse. This electric pulse response is transmitted to the processor that identifies the position to which the light pen is pointing. As with cursor-positioning devices, recorded light-pen coordinates can be used to position an object or to select a processing option.

It facilitates drawing images and selects objects on the display screen by directly pointing the objects with the pen.

Although light pens are still with us, they are not as popular as they once were since they have severage disadvantages compared to other input devices that have been developed.

- A light pen is pointed at the screen; part of the screen image is obscured by the hand and pen.
- Prolonged use of the light pen can cause arm fatigue. Also, light pens require special implementationfor some applications because they cannot detect positions within black areas. To be able to select positions in any screen are with a light pen, we must have some non-zero intensity assigned to each screen pixel.
- Light pens sometime give false readings due to background lighting in the room.

Touch screen

It is an easisest way to enter data with the tough of a finger. Touch screens enable the user to select an

option by pressing a specific part of the screen. Touch input can be recorded using optical, electrical or acoustical methods.

Infrared (optical Touch sensitive Screen)

- An infrared touch screen uses an array of X-Y infrared LED and photo detector pairs around the edges of the screen to detect a disruption in the pattern of LED beams. A major benefit of such a system is that is can detect essentially any input including a finger, gloved finger, stylus or pen. It is generally used in outdoor applications and point-of-sale systems which can't rely on a conductor (such as a bare finger) to activate the touch screen. Unlike capacitive touch screens, infrared touch screens do not require any patterning on the glass which increases durability and optical clarity of the overall system.

Resistive (Electrical Touch Sensitive Screen)

A resistive touch screen panel is composed of several layers, the most important of which are two thin, metallic, electrically conductive layers separated by a narrow gap. When an object, such as a finger, presses down on a point on the panel's outer surface the two metallic layers become connected at that point: the panel then behaves as a pair of voltage dividers with connected outputs. This causes a change in the electrical current, which is registered as a touch event and sent to the controller for processing.

Surface acoustic wave

Surface acoustic wave (SAW) technology uses ultrasonic waves that pass over the touch screen panel. When the panel is touched, a portion of the wave is absorbed. This change in the ultrasonic waves registers the position of the touch event and sends this information to the controller for processing. Surface wave touch screen panels can be damaged by outside elements. Contaminants on the surface can also interfere with the functionality of the touch screen.

Graphics tablet

A graphics tablet consists of an electronic writing area and a special "pen" that works with it. Graphics tablets allows artists to create graphical images with motions and actions similar to using more traditional drawing tools. The pen of the graphics tablet is pressure sensitive, so pressing harder or softer can result in brush strokes of different width (in an appropriate graphics program).

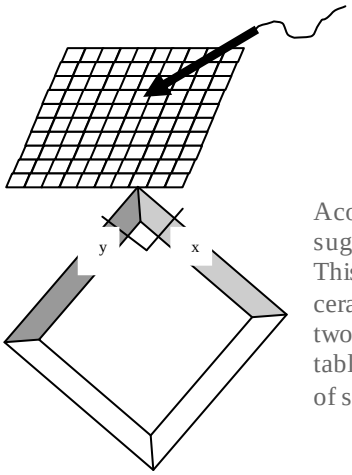
A graphics tablet is an input device used by artists which allows one to draw a picture onto a computer screen without having to utilize a mouse or keyboard. A graphics tablet consists of a flat tablet and some sort of drawing device, usually either a pen or stylus. A graphics tablet may also be referred to as a drawing tablet or drawing pad. While the graphics tablet is most suited for artists and those who want the natural feel of a pen-like object to manipulate the cursor on their screen, non-artists may find them useful as well. The smooth flow of a graphics tablet can be refreshing for those who find the mouse to be a jerky input device, and repetitive stress injuries such as carpal tunnel syndrome are less likely when using a graphics tablet.

These devices are more accurate than light pens.

Based on the mechanism used to find two-dimensional coordianes on a flat surface, there are two types of tablets: Electromagnetic Field and Acoustic tablet.

Electromagnetic Field or Voltage Tablet

These types of tablets are constructed with a rectangular grid of wire embedded in the tablet surface, with different voltages or magnetic fields corresponding to different coordinates. Electromagnetic pulses are generated in sequence along the wires, and



an electrical signal is induced in a wire coil in an activated stylus or hand cursor to record a tablet position. Depending on the technology, and their signal strength, coded pulses, or phase shifts can be used to determine the position on the tablet.

Acoustic or Sonic Tablet

Acoustic tablet designed by the Science Accessories Corporation works on the acoustic principle suggested by Brenner.
This type of tablet uses sound waves to detect a stylus position. The stylus has a small piece of ceramic mounted at its tip; a small spark is generated across the surface of the ceramic between two electrodes. The sound of the spark is picked by the strip microphones along the edges of the tablet. The perpendicular distances of the stylus tip from the axes is proportional to time intervals of sound created and received at destination.

Scanners

A scanner is a device that images a printed page or graphic by digitizing it, producing an image made of tiny pixels of different brightness and color values which are represented numerically and sent to the computer. Scanners scan graphics, but they can also scan pages of text which are then run through OCR (Optical Character Recognition) software that identifies the individual letter shapes and creates a text file of the page's contents.

Microphone

A microphone can be attached to a computer to record sound (usually through a sound card input or circuitry built into the motherboard). The sound is digitized—turned into numbers that represent the original analog sound waves—and stored in the computer to later processing and playback.

MIDI Devices

MIDI (Musical Instrument Digital Interface) is a system designed to transmit information between electronic musical instruments. A MIDI musical keyboard can be attached to a computer and allow a performer to play music that is captured by the computer system as a sequence of notes with the associated timing (instead of recording digitized sound waves).

GRAPHICAL USER INTERFACE AND INTERACTIVE INPUT METHODS

The User Dialogue:

For a particular application, the user's model serves as the basis for the design of the dialogue.

User model describes

- The purpose of the system
- Graphic operations available.

(e.g)

1. Architectural design tool.
   The model describes the construction methods using the package and displays views of the buildings by positioning walls, doors, windows etc.

2. Facility Layout System.
   Objects = set of furniture items.
   Operations = positioning and removing different pieces of furnitures within the facility layout.

All information in the user dialogue are presented in the language of the application.

General Considerations:

1. **Windows And Icons.**

   Window manager interface is exising in the window system. The system also provides functions for handling the display & manipulation of the windows.
   (e.g) resizing, relocating, opening & closing windows.

   Display routines provide the interior and exterior clipping and other graphics functions. Windows have sliders, buttons, menu icons for selecting various window options. X Windows & NeWs are capable of supporing multiple window managers.

   Icons representing objects such as furniture items and circuit elements are often referred to as **application icons**.

   Icons representing actions, such as rotate, magnify, scale, clip and paste are called **control icons/command icons**.

2. **Accommodating Multiple Skill Levels.**

   Interactive graphical interfaces provide several methods for selecting actions. Options can be selected by pointing at an icon, and clicking different mouse buttons, accessing pull-down or pop-up menus or by typing keyboard commands. This allows a package to accommodate users that have different skill levels.

   Inexperienced user : Interface with

- Few easily understood operations
- Detailed prompting
- A simplified set of menus and icons is easy to learn and remember.
- Simple point and click operations are easier.

Interfaces typically provide a means for masking the complexity of a package to be used easily for the beginners.

Experienced Users:

- Need speed.
- Fewer prompts.(more input from the keyboard and multiple mouse button clicks).
- Actions selected with function keys / simultaneous combination of keyboard keys.

3. Consistency.
   An important design consideration is consistency. A particular icon shape should always have a single meaning, it should not change depending on actions or objects incontext.

   Placing menus in the same relative positions, reducing the hunt for the user.

   The objects and operations provided should be designed to form a minimum and consistent set so that the system is easy to learn, but not oversimplified to the point where it is difficult to apply.

4. Minimizing memorization
   Operations should be easy to understand and to remember. Complicated, inconsistent and abbreviated command formats lead to confusion and reduction in the effectiveness of the use of the page.
   (e.g) one key or button for all delete operations will be easy to handle and is easier to remember.

5. Backup & Error Handling

   An operation can be called before execution is completed returning back to the state it was in before the operation started.

   With the ability to back up, we can confidently explore the capabilities of the system, knowing that the effects of a mistake can be erased.

6. Feedback.
   a. To inform of actions in progress at each step, when the response time is high.
   b. Object highlighting, icon, a message are examples of feedback.
   c. Several feedback messages can be displayed to inform the current status.
   d. Feedback can be given as audible click or by lighting up the key that was pressed (for function keys).
   e. Audio feedback is advantageous, since it does not occupy space.

f. Invert pixel intensities, highlighting, blinking and color changes.
g. A cross, thumbs-down symbol, blinking "at work" indicators. These are effective for more experienced users.
h. "echo" feedback is desirable. Typed characters can be displayed on the screen as they are input to detect and correct errors.
i. Scalar values selected with dials are echoed. Selection of coordinate points are echoed.

INPUT OF GRAPHICAL DATA

The GKS input model is based on the concept of logical input devices. Logical input devices provide the application program with an interface which abstracts physical input devices from a particular hardware configuration.

A logical input device consists of

a. Class.

The class of a logical input device defines the type of the input value which is returned. The six different classes are given in the following table :

The GKS logical input classes.

Device	Returntype
Locator	Wc, tran
Choice	Choice
Pick	Pickid
Valuator	Value
String	String
Stroke	Wc [1, 2, …n], ntran

The actual number of logical devices in each class is workstation dependent. Each individuallogical input device within a class is distinguished by a unique number.

b. Mode.

The activation mode indicates how the input value is obtained from the logical input device. Conceptually, there are always two processes running for each active logical input device; these are the so-called *measure process* and *trigger process*. A particular measure value of a logical input device is defined to be the (eventually transformed to world coordinates) value of the physical input device.

The measure process will always contain the current measure value of the logical input device. Usually, the measure value is echoed in some way on the screen, (for instance, by echoing a cursor shape in the position that corresponds with the measure value).

A trigger process is an independent, active process that, when *triggered* by the user, sends a message to the measure process. Triggering a logical input device indicates that the current measure value must be returned to the application.

How the measure value is mapped onto a value returned by a logical input device is defined differently for every input class. For the *locator* device the mapping rules are:

♦ Transform the measure value (given in device coordinates) back to normalized device coordinates using the inverse of the current workstation transformation.

• Select the normalization transformation with the highest viewport input priority in whose viewport the normalized coordinate lies. The selection of a normalization transformation will always succeed since there is a default normalization transformation which covers the complete normalized device coordinate space.

♦ transform the normalized coordinate back to a world coordinate using the inverse of the selected normalized transformation.

♦ return the world coordinate and the number of the selected normalization transformation to the application program.

There are three different activation modes:

♦ ***request***

In the case of request mode, the application program will wait until the trigger process sends a message to the measure process. The value of the measure process at the moment of triggering will then (after the necessary transformations) be passed to the application program.

♦ ***sample***

In the case of sample mode, the value of the measure process will, at the moment of sampling, be passed to the application program. No triggering is involved when a logical device is sampled so that the application program will immediately continue after issuing a sample call.

♦ ***event.***

In the case of event mode, the application program will not wait until the trigger process sends a message to the measure process. However, when the logical input device is triggered the value of the measure process at the moment of triggering is put in an input queue. The contents of the queue can be acquired by the application program by issuing calls that query and get the queue elements.

GKS organizes data that can be input to an applications program into six types, each related to a *Logical Input Device*. The actual physical input devices are mapped onto these logical devices, which makes it possible for GKS to organize the different forms of data in a device-independent way, and thus helps to make the code more portable. A logical input device is identified by 3 items:

1. a workstation identifier
2. an input class
3. a device number

The six input classes and the logical input values they provide are:

LOCATOR

Returns a position (an x,y value) in World Coordinates and a Normalization Transformation number corresponding to that used to map back from Normalized Device Coordinates to World Coordinates. The NT used corresponds to that viewport with the highest *Viewport Input Priority* (set by calling GSVPIP). **Warning:** *If there is no viewport input priority set then NT 0 is used as default, in which case the coordinates are returned in NDC.* This may not be what is expected!

```
CALL GSVPIP(TNR, RTNR, RELPRI)
```

TNR
Transformation Number
RTNR
Reference Transformation Number
RELPRI
One of the values 'GHIGHR' or 'GLOWER' defined in the Include File, ENUM.INC,

Examples : mouse, joystick, trackball,s paceball, hand cursor, keyboard – 4 way / 8 way arrow keys.

STROKE

To input a sequence of coordinate positions.

Stroke Device = Multiple calls to a locator device.

➔ Continuous movement of mouse, trackball, joystick or tablet hand cursor is translated into a series of input coordinate values.

Returns a sequence of (x,y) points in World Coordinates and a Normalization Transformation as for the Locator.

Example : “graphics tablet”. Button activation can be used to place the tablet into “continuous” mode.

VALUATOR

To input scalar values, valuators are used for setting various graphics parameters, such as rotation angle and scale factors and physical parameters. (temperatures, voltage levels etc).

Control dials – rotate the dial. Rotary Potentiometer converts dial rotation into corresponding voltage.

Keyboard – type the value.

Joysticks, trackballs, tablets and other interactive devices can be adapted for valuator input by interpreting pressure or movement of the device relative to a scalar range.

Returns a real value, for example, to control some sort of analogue device.

CHOICE

Menus are used to select programming options, parameter values and object shapes to be used in constructing a picture. A choice device is defined as one that enters a selection fro m a list of alternatives.

Buttons can be programmed. When a coordinate position (x,y) is selected, it is compared to the coordinate extents of each listed menu item. A menu item with vertical and horizontal boundaries at the coordinate values x_{min}, x_{max}, y_{min} and y_{max} is selected if the input coordinates (x,y) satisfy the inequalities.

$x_{min} <= x <= x_{max}$
$y_{min} <= y <= y_{max}$

Returns a non-negative integer which represents a choice from a selection of several possibilities. This could be implemented as a menu, for example.

STRING

Keyboard is the primary device. Input characters are typed. Other devices can also be used to input character patterns in a "text writing" mode.

Returns a string of characters from the keyboard.

PICK

Graphical object selection is the function of this logical class of devices. They are used to select parts of a scene that are to be transformed or edited in some way. = cursor positioning device.

With a mouse or joystick, ew can position the cursor on the primitives in a displayed structure and press the selection button.

The position of the cursor is then recorded, and several levels of search are necessary to locate the particular object.

- ➔ The cursor position is compared to the coordinate extents of various structures in the scene.

➔ If two or more structures areas contain the cursor coordinates, further checks are necessary. The coordinate extends of each individual line is checked next.

Returns a segment name and a pick identifier of an object pointed at by the user. Thus, the application does not have to use the locator to return a position, and then try to find out to which object the position corresponds.

Chapter 9: Input Classes

9.1 INTRODUCTION

The functionality in PHIGS which supports graphical input is described, together with the mechanisms which relate graphical input to the central structure store.

PHIGS shares the same model of input as GKS, and readers familiar with GKS will note that the differences lie in the choice of measure values for the input device classes.

Input in PHIGS is accomplished through logical input devices. The idea behind logical input devices is that they provide an abstraction from specific hardware input devices, for example, mouse, tablet, keyboard, lightpen, and enable applications to be written in such a way that they are portable across a wide range of input device hardware. The application is only concerned with the values delivered by logical input devices, for example a position in world coordinates and view index, not with the specific details of how the value is derived from the values generated by the physical input devices.

PHIGS provides six classes of logical input devices. The types of values returned by each class are:

1. LOCATOR: a position in world coordinates and a view index;
2. STROKE: a sequence of positions in world coordinates and a view index;
3. VALUATOR: a real number;
4. CHOICE: a CROICE status and an integer representing selection from a set of choices;
5. PICK: a PICK status and pick path;
6. STRING: a character string.

As in GKS, there are three operating modes for logical input devices. They differ in whether the operator or the application has the initiative to generate logical input values. The three operating modes are:

1. REQUEST: input is produced by the operator in direct response to the application;
2. SAMPLE: input is acquired directly by the application;
3. EVENT: input is generated asynchronously by the operator and is collected in a queue for the application.

PHIGS allows the application to control some characteristics of logical input devices, for example the forms of prompting to be used to indicate to the operator that input is required. The logical input device classes and operating modes are explained in detail in the following sections.

9.2 REQUEST MODE

The default operating mode, is REQUEST. The logical input device classes are then described for REQUEST mode input.

In REQUEST mode, input is produced by the operator in direct response to a request by the application program. This corresponds very closely to the forms of input provided by programming languages such as Fortran, Pascal and C. In Fortran, when a READ statement is executed, the program is suspended and waits for the values requested to be provided by the input sub-system. Execution of the Fortran program continues when the values have been provided. This is exactly the effect of REQUEST mode input in PHIGS. The application requests input from a particular logical input device. The application is then suspended until the logical input device has obtained the input from the operator. In this mode, either the application is active or the logical input device is active, but never both. A consequence of this is that never more than one logical input device can be active at a time in REQUEST mode.

The form of the function for REQUEST input is:

REQUEST XXX(WS, DV, ST, ...)

XXX identifies the class of logical input device (VALUATOR, CHOICE, etc), WS and DV identify the particular device of that class from which input is requested and ST is an output parameter which is a status indicator providing information on the completion of the request. WS identifies the workstation with which the logical input device is associated and DV identifies the particular device of the class on that particular workstation. It is possible for a workstation to provide more than one logical input device in a class, thus a workstation might provide three LOCATOR devices.

The form of the remaining parameters to the REQUEST function varies between the different logical input device classes, and is described in the following sections.

9.3 LOCATOR

The LOCATOR input device returns a single position to the application program together with information that relates the point on the display with the view associated with this point. Two functions are provided in PHIGS to request input from a LOCATOR device:

REQUEST LOCATOR 3(WS, DV, ST, VI, XPOS, YPOS, ZPOS)
REQUEST LOCATOR (WS, DV, ST, VI, XPOS, YPOS)

REQUEST LOCATOR 3 returns a position (XPOS,YPOS,ZPOS) in world coordinates and a view index VI. REQUEST LOCATOR returns a 2D position in world coordinates (XPOS,YPOS) and a view index VI. The 2D position is obtained by discarding the Z-coordinate of the 3D position which is the intrinsic value of the LOCATOR logical input device irrespective of how it is invoked.

It is worth stressing that all LOCATOR devices in PHIGS are 3D devices. The REQUEST LOCATOR 3 and REQUEST LOCATOR functions can be used to access any LOCATOR device.

To generate an input value in world coordinates from a value in the workstation dependent device coordinate system, it is necessary to apply the inverse of the workstation and viewing transformations. Each workstation has a single workstation transformation (see Section 11.3), but may have multiple viewing transformations. The parameter VI identifies which of the viewing transformations was used to transform the logical input value. This transformation will be one whose view clipping limits contain the position. The mechanism for selecting the viewing transformation is discussed shortly, after an example has been described.

The major difficulty with this style of programming is to relate LOCATOR input in world coordinates to the modelling coordinate systems in which the components of a scene are specified. In the worst case this can involve the application in considerable work maintaining details of the coordinate systems used and the transformations between them.

9.3.1 Multiple viewing transformations

Typically, several views will be in use on a workstation, for example in Figure 9.3 the desk is drawn with view index 1, and the word STOP with view index 2.

The operator needs to be able to input LOCATOR positions in either view and the application program needs to be able to differentiate between the views in use. This is the purpose of the view index parameter of the REQUEST LOCATOR function.

The view transformation selected to transform the LOCATOR position from NPC to world coordinates is that whose view clipping limits contain the position in NPC coordinates.

9.4 STROKE

Whereas the LOCATOR logical input device returns a single position in world coordinates and a view index, to the application program, the STROKE logical input device returns a sequence of positions and a view index. Two PHIGS functions are provided to request input from a STROKE device:

REQUEST STROKE 3 (WS, DV, N, ST, VI, NPTS, XPOSA, YPOSA, ZPOSA)
REQUEST STROKE (WS, DV, N, ST, VI, NPTS, XPOSA, YPOSA)

The parameters WS, DV, ST and VI have the same meanings as the corresponding parameters in REQUEST LOCATOR 3 and REQUEST LOCATOR. The parameter N is the dimension of the arrays XPOSA, YPOSA (and ZPOSA). The output parameter NPTS returns the number of points in the STROKE value input, and the remaining parameters are arrays of coordinate values which contain the coordinates of the points in the STROKE. The provision of STROKE input as well as LOCATOR input is to allow a sequence of positions to be input without placing too much load on the PHIGS system and, in consequence, producing a faster response to the operator's input of positions.

All the points in the STROKE are transformed by the inverse of the view transformation corresponding to view index VI. The view transformation selected is that with highest view transformation input priority whose view clipping limits contain all of the points in the STROKE.

9.5 2D INPUT DEVICES

LOCATOR and STROKE input are conceptually 3D in PHIGS. If the physical input device generating LOCATOR and STROKE values is a 2D device, a third coordinate value is appended either internally from the PHIGS state tables, or externally, for example by requesting the operator to type the Z-value from a keyboard device.

9.6 LOCATOR AND STROKE IN 3D

LOCATOR 3 and STROKE 3 logical input values are obtained by applying the inverse of the workstation transformation and the inverse of the selected view transformation.

In 3D, it is possible that these transformations do not have inverses. These situations are handled as follows. If the workstation transformation does not have an inverse, the Z-component of the LOCATOR position, and the Z-components of all the STROKE positions are set to the minimum Z-value of the workstation window.

In the case of the view transformation, the only view transformations considered are those which are invertible. Any view transformation, which does not have an inverse, is rejected. View transformation 0 is invertible, so it is guaranteed that a view transformation can always be found which can back transform LOCATOR and STROKE positions from NPC to world coordinates.

9.7 PICK

The PICK logical input device provides a link between the image displayed on a workstation and the central structure store. The operator can pick a portion of the display. The value returned by the pick logical input device identifies the structure element in the central structure store which generated the picked primitive. The PHIGS function to request input from a PICK device is:

REQUEST PICK(WS, DV, DEPTH, ST, PPD, PP)

DEPTH is an input parameter which specifies the maximum depth of pick path to return. PPD is the depth of the actual pick path returned and PP is the pick path which identifies the primitive picked. This will be explained in the example below.

A pick path defines the traversal path through the central structure store, which generated the output primitive picked. A pick path is a list of items, each item consisting of:

- structure identifier
- pick identifier
- element position

Pick identifiers will be described shortly.

9.7.1 Pick identifier

Another method for differentiating between several instances of a primitive is to give the instances different pick identifiers. A pick identifier is a primitive attribute which provides an additional level of naming of primitives for the application program to use. The pick identifier attribute is bound to a primitive on creation. The function:

SET PICK IDENTIFIER(ID)

creates a structure element in the currently open structure, which, on traversal, results in the value ID being bound to subsequently created output primitives until a further set pick identifier structure element is traversed.

9.8 VALUATOR

The VALUATOR logical input device returns a real value to the application. The function to request input from a VALUATOR logical input device is:

REQVEST VALUATOR(WS, DV, ST, VAL)

The application can specify the range within which the value may be; the mechanism for achieving this is described in a later section (Section 10.2.4).

The most natural physical input device to map onto the VALUATOR logical input device would be a potentiometer, but in common with the other logical input devices, the logical device can be supported by a wide range of physical input devices. PHIGS input devices are most likely to support input of values from a mouse at the very least. Many workstations provide a set of dials which will normally be mapped into a set of VALUATORs. At the other extreme, inputting the value from a keyboard may be provided.

9.9 CHOICE

The CHOICE logical input device returns an integer (greater than 0) which indicates which of a number of possibilities has been chosen by the operator. The PHIGS function to invoke a CHOICE logical input device in REQUEST mode is:

REQUEST CHOICE(WS, DV, ST, CH)

where CH returns the integer representing the selection made.

The example in the previous section showed how a VALUATOR logical input device could be used to allow the operator to define the orientation of an object, in this case the lamp base in the work environment. The structure network which models the lamp also allows the arm and light to be rotated A program which allows the operator to select the lamp, arm or light and then define the orientation of the component selected, could use a CHOICE logical input device to select the component, and a VALUATOR logical input device

CHOICE logical input devices can be realized by many different physical input devices. One obvious way to implement a CHOICE device is as a menu of items selectable by a mouse. The application program needs to be able to associate menu items with choice numbers; the mechanism for doing this is described when the logical input device initialization is discussed.

9.10 STRING

The STRING logical input device returns a character string to the application program. The PHIGS function for requesting input from a STRING logical input device is:

REQUEST STRING(WS, DV, ST, NCHARS, STR)

STR returns the character string that was input and NCHARS returns the number of characters it contains. STRING input is useful for inputting items such as filenames and labels.

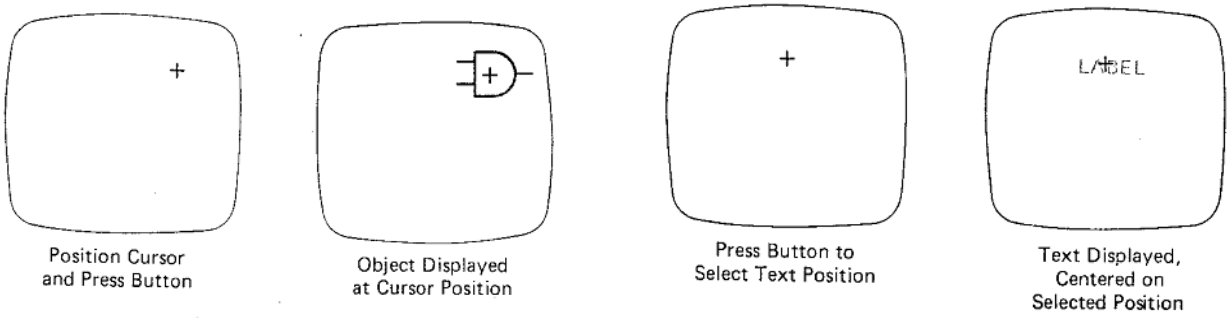
STRING logical input devices would normally be mapped to a physical keyboard but hand-drawn characters or clicking with a mouse on a displayed pseudo-keyboard would be equally valid. As with the STROKE device, it is necessary to define the completion of the input by specifying an event which terminates the input.

INTERACTIVE PICTURE CONSTRUCTION TECHNIQUES.

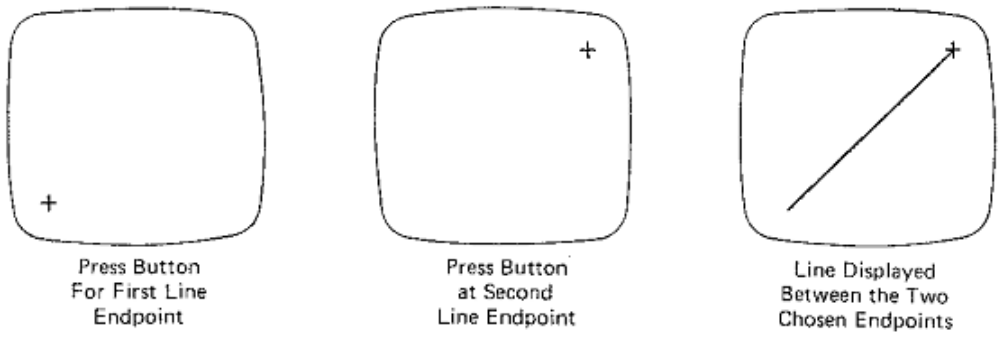
- ➔ basic positioning methods.
- ➔ Constraints
- ➔ Grids
- ➔ Gravity fields
- ➔ Rubber-band methods
- ➔ Sketching
- ➔ Dragging

1. **Used to specify a location for an object or a character string**

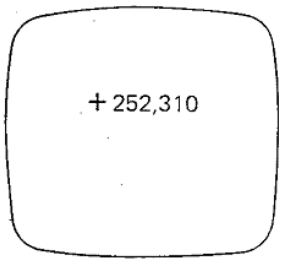
   - ➔ The cursor is moved to the desired location.
   - ➔ A button is pressed to fix the object at this location.



2. **Used to draw lines**



3. **Used to place the cursor at a predetermined position**

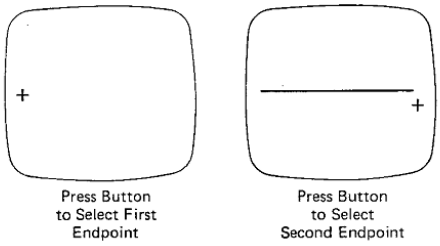


CONSTRAINTS

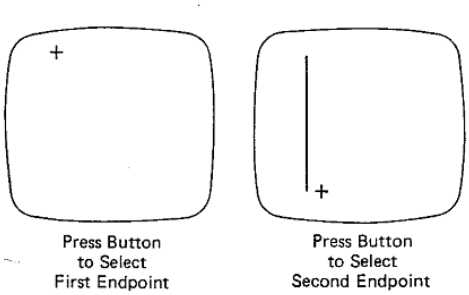
Used to achieve predetermined orientations and alignments.

Common constraints

Horizontal alignment

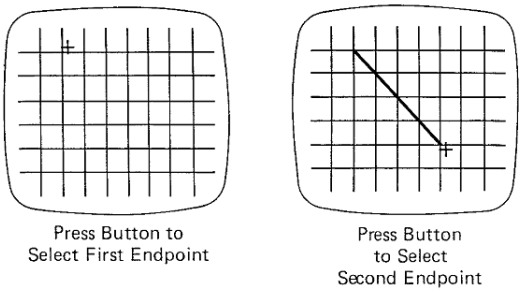


Vertical alignment



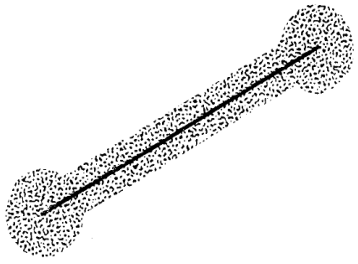
GRIDS

- → Used to round coordinate positions to the nearest positions to the nearest grid intersection.
- → useful for positioning and aligning objects and text.
- → grids can be displayed or invisible.



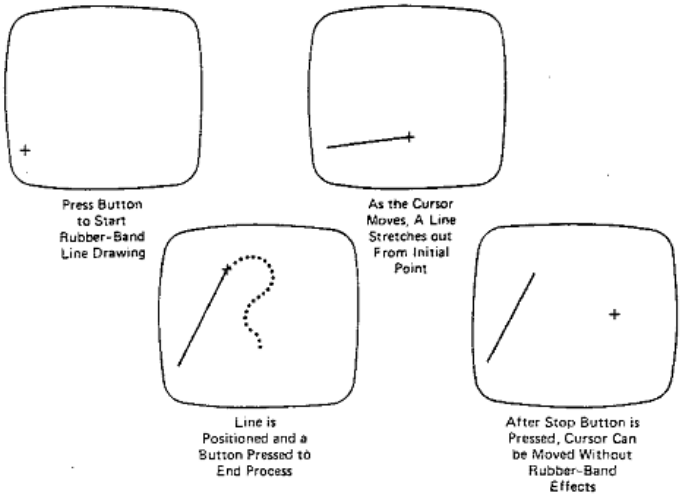
GRAVITY FIELDS

- ➢ Used to connect a new line to a previously drawn line.
- ➢ Normally the gravity field is not displayed.

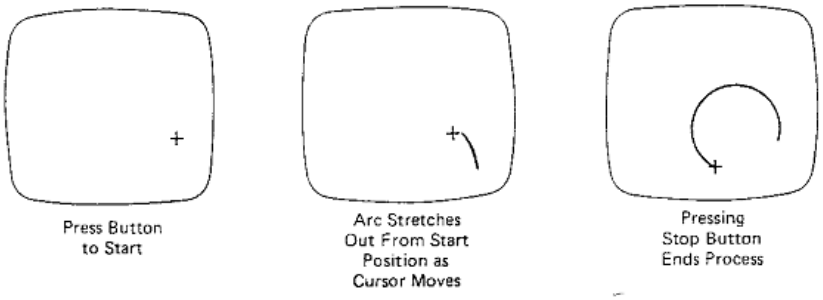


Rubber-band methods

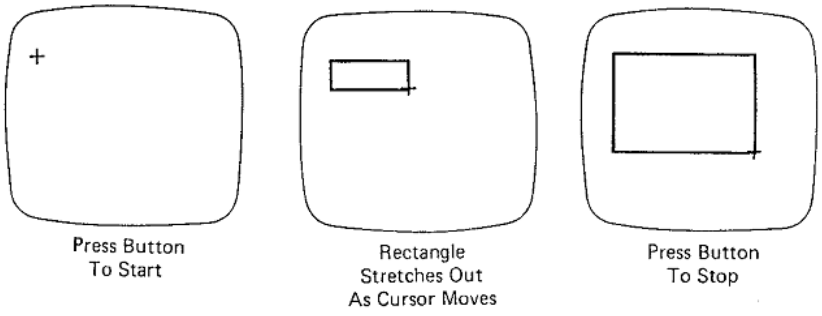
- ✓ Used to construct and position straight lines



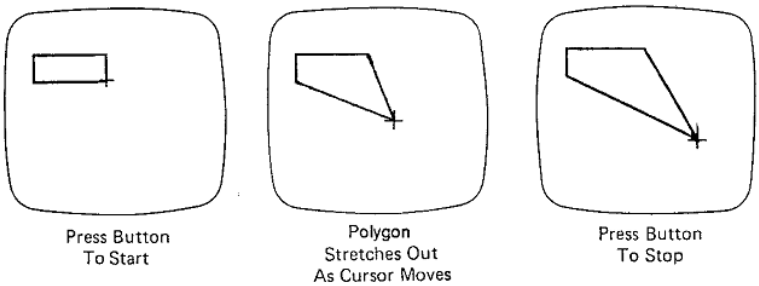
✓ Used to construct circular arcs



✓ Used to scale objects

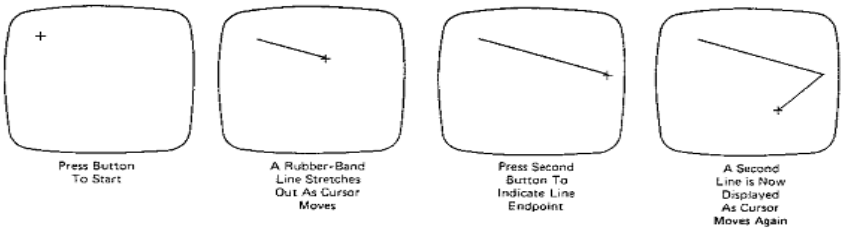


✓ Used to distort objects by allowing only the line segments attached to a single vertex to change

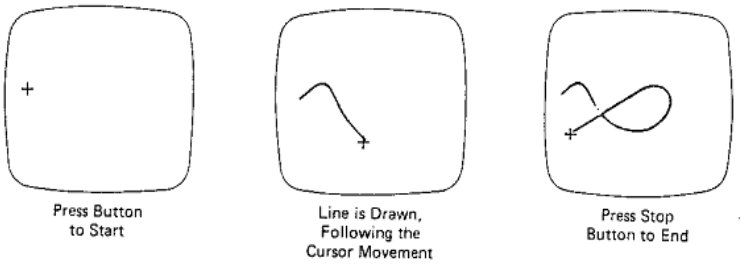


SKETCHING

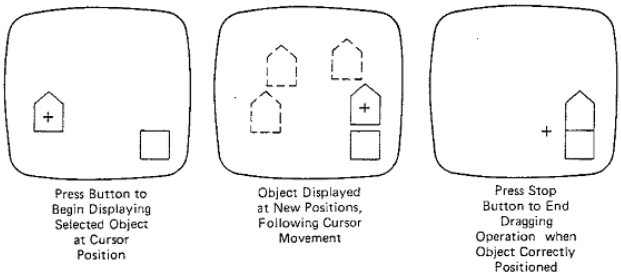
- ✓ Uses rubber-band methods to create objects consisting of connected line segments



- ✓ Uses stroke techniques to create curved figures. A variety of brushes can be provided
  - Different thickness
  - Different textures
  - Different colors (including background)
  - Even patterns.



- ✓ Dragging
  - Used to reposition objects
    - Select an object from the menu
    - Position the object
    - Release the object.



CHAPTER – 2

BASIC TRANSFORMATIONS

Simple transformations such as translation, rotation and scaling of objects and vectors can be performed using matrices.

Translation

Translation simply moves an objects position by given offset. This is performed by adding the translation vector to the all co-ordinates within the object. For example, in figure 8(a) the box is positioned with its bottom left corner at point (1.5, 0.5). The size of the box is 2 units wide and 1 unit tall. If we translate this box by the vector (-0.5, 2.0) which is illustrated by the blue arrow, we see in figure 8(b) that the box moves to its new position of (1.0, 2.5) which is obtained by simply adding the translation vector to each set of co-ordinates. Note that the box's size and orientation remain unchanged.

Expressing this mathematically, we want to find the new co-ordinates (x', y') from the original co-ordinates (x, y) by applying the translation (t_x, t_y). This is simply :

$$x' = x + t_x$$
$$y' = y + t_y$$

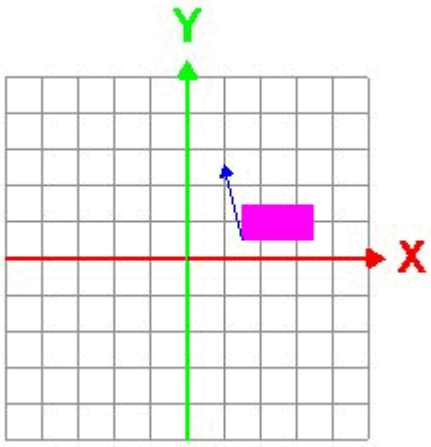


Figure 8(a). Original position of box.

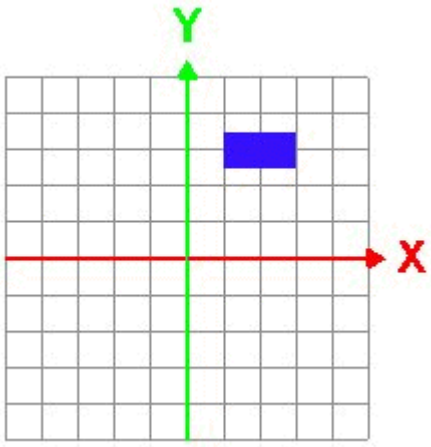


Figure 8(b). Box after translation of (-0.5, 2.0).

Scaling

Scaling adjusts the positions of all the points in an object by the same factor and as a result it alters both the size *and* position of an object. If we look at the same box as before, shown again in figure 9(a). This time we apply a scaling transformation of (0.5, 3.0). All the co-ordinates of the box have their x-values scaled by a factor of 0.5 and their y-values scaled by a factor 3.0. This results in the final position and size shown in

Figure 9(b). Note that the orientation of the box has not changed, it has been made narrower and taller.

Again, expressing this mathematically to obtain the new co-ordinates (x', y') from the original co-ordinates (x, y) by applying the scaling factor (S_x, S_y).

$$x' = x.S_x$$
$$y' = y.S_y$$

To make this even more convenient we can express the transformation as a matrix multiplication as follows. Looking at the equation below and it's three matrices, the **vector** (right) is multiplied by the **transformation matrix** (centre) to produce the **resultant vector** (left).

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Note that in the examples above we are scaling using the origin (0, 0) as the centre of the scaling.

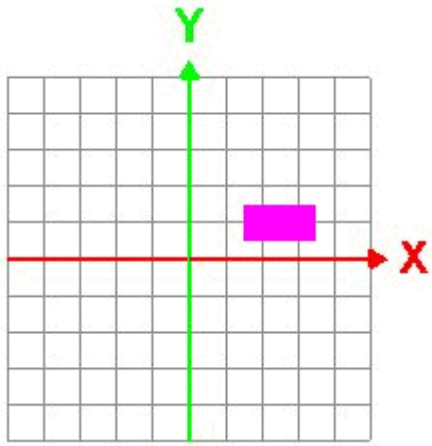


Figure 9(a). Original position of box.

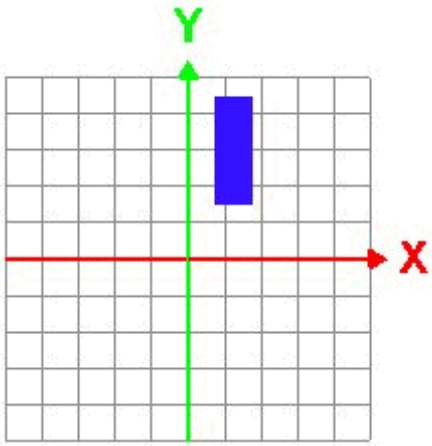


Figure 9(b). Box after scaling by (0.5, 3.0).

Rotation

Rotation changes the position and orientation of shapes by rotating each of the points in the shape around a common point, or centre of rotation. For simplicity we will use the origin as the centre of rotation. Figure 10(a) shows the original box from the previous two examples. Figure 10(b) shows the result of a rotation of 45° counter clockwise about the origin. Note that the actual shape or size of the box has not changed, only it's position and orientation.

The new positions of each of the points after a rotation through an angle θ about the origin can be found using :

$$x' = x.\cos\theta - y.\sin\theta$$
$$y' = x.\sin\theta + y.\cos\theta$$

Note that the new positions for x' and y' now depend on *both* the values for the previous x and y. As with scaling transformations, a rotation operation can be expressed using matrices.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

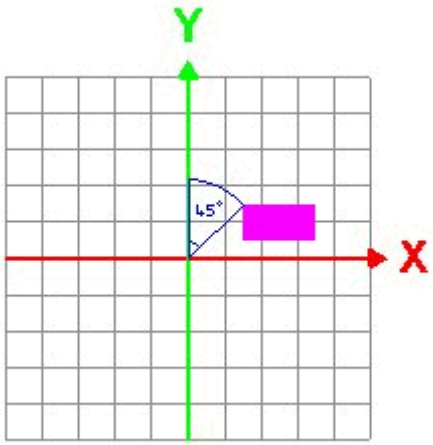


Figure 10(a). Original position of box

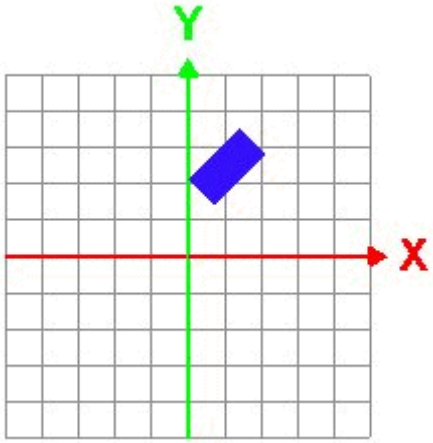


Figure 10(b). Box after rotation 45° counter clockwise

Homogeneous co-ordinates

The previous section showed how we could mathematically represent the transformations of translation, scaling and rotation. Both scaling and rotation can be represented very conveniently using matrices, but unfortunately translation cannot. It would be nice to be able to perform all transformations using the same operation, i.e. by multiplying a position vector by a transformation matrix. This is where homogeneous co-ordinates come in useful.

Homogeneous co-ordinates add an extra scaling co-ordinate (typically w) to a normal 2D or 3D vector. In order to convert between homogeneous co-ordinates and normal co-ordinates we must take this scaling factor into consideration.

Normal 2D vector $\begin{bmatrix} x \\ y \end{bmatrix}$

Homogeneous 2D vector $\begin{bmatrix} x \\ y \\ w \end{bmatrix}$

Homogeneous 3D vector $\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$

In homogeneous co-ordinates, 2 points $\begin{bmatrix} x \\ y \\ w \end{bmatrix}$ and $\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix}$ are only equal if $\frac{x'}{w'} = \frac{x}{w}$ and $\frac{y'}{w'} = \frac{y}{w}$.

To convert a normal vector into homogeneous co-ordinates, simply add the additional "*w*" co-ordinate with a value of 1. To convert a vector using homogeneous co-ordinates into a normal vector, divide each of the values by the scaling factor *w*.

A single point in normal co-ordinate space can be represented by many points in homogeneous co-ordinates. For example :

$$\begin{bmatrix} 1 \\ 3 \end{bmatrix} \equiv \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix} \equiv \begin{bmatrix} 2 \\ 6 \\ 2 \end{bmatrix} \equiv \begin{bmatrix} 7 \\ 21 \\ 7 \end{bmatrix} \equiv etc...$$

Finally, if the scaling factor, *w*, is zero then the point is at infinity.

If we now return to our transformations and see how we can represent them using homogeneous co-ordinates and matrix operations.

Translation

Taking the problem of translation again, now that we are using homogeneous co-ordinates to represent the vectors it is possible to use matrices to perform the transformation. The transformation matrix has to be 3*3 in size now, due to the homogeneous co-ordinates being represented as 3 value vectors. The 2D translation values are given again as t_x and t_y.

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix} \equiv \begin{cases} x' = x + w.t_x \\ y' = y + w.t_y \\ w' = w \end{cases} \equiv \begin{cases} \frac{x'}{w'} = \frac{x}{w} + t_x \\ \frac{y'}{w'} = \frac{y}{w} + t_y \end{cases}$$

Scaling

The scaling and rotation transformations could already be performed using matrix multiplication, however we need to adjust the transformation matrix to account for the homogeneous co-ordinates. As before, the scaling values are given by S_x and S_y.

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix} \equiv \begin{cases} x' = x.S_x \\ y' = y.S_y \\ w' = w \end{cases} \equiv \begin{cases} \frac{x'}{w'} = S_x \frac{x}{w} \\ \frac{y'}{w'} = S_y \frac{y}{w} \end{cases}$$

Rotation

The rotation transformation is performed using matrices and homogeneous co-ordinates as follows. As before, the rotation angle is represented by θ and the centre of rotation is the origin.

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix} \equiv \begin{cases} x' = x.\cos\theta - y.\sin\theta \\ y' = x.\sin\theta + y.\cos\theta \\ w' = w \end{cases} \equiv \begin{cases} \frac{x'}{w'} = \frac{x}{w}\cos\theta - \frac{y}{w}\sin\theta \\ \frac{y'}{w'} = \frac{x}{w}\sin\theta + \frac{y}{w}\cos\theta \end{cases}$$

Compound transformations

We have just seen how we can use various transformation matrices to apply rotations, translations and scaling operations. Each of these transformations is represented as a 3*3 matrix (or 4*4 in the case of 3D co-ordinates). We can combine any set of transformations by simply multiplying the individual matrices together.

Any transformation may be expressed as a matrix. For example, if we wanted to perform a scaling operation (S) followed by a translation (T) we simply multiply the two transformation matrices together to give a 3*3 compound matrix which will perform both operations. This is shown below.

$$\begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \equiv \begin{bmatrix} S_x & 0 & S_x.t_x \\ 0 & S_y & S_y.t_y \\ 0 & 0 & 1 \end{bmatrix}$$

As we mentioned before, matrix multiplication is not commutative and the order of application of the transformations is vitally important. A different order will produce very different results. The only exception to this rule is that the order is not important if

the transformations are of the same type. I.e. a translation followed by a translation, a rotation followed by a rotation, etc.

An example of one such use of compound transformations is to provide more generic transformations, e.g. rotation about an arbitrary point. A common method for achieving this is to compound a number of standard transformations to achieve the same result. For example, if we wanted to rotate an object about a point P (we will denote this operation as R_P) we would perform the following series of transformations all of which can be concatenated into a single transformation matrix. Firstly we must translate the object from point P to the origin with the translation T_{-P}. We then rotate the object about the origin (R_O) using the standard rotation transformation. Finally, we translate the object back to the point P using the translation T_P. So, we can create a transformation R_P which can rotate about any point P as follows : $R_P = T_{-P}R_OT_P$.

Line drawing algorithms

Introduction

The raster grid consists of discrete cells called pixels which are referenced by integer screen coordinates. In order to display a geometric primitive, the intensities of pixels very close to the desired primitive are loaded into the frame buffer in the form of bits. The bits corresponding to each pixel are converted into analog values, which illuminate phosphor dots representing the pixels on the raster grid. The coordinates of actual points on any primitive need not be integers whereas we require the integer coordinates.

Thus there is a need to scan convert the geometric primitive such that the screen positions approximate the actual coordinates of the primitives.

The process of determining which pixels provide the best approximation to the desired line is properly known as **rasterization.**

When combined with the process of generating the picture in scan line order, it is known as **scan conversion.**

Line Drawing

- Draw a line on a raster screen between two points.
- What's wrong with statement of problem?
  - doesn't say anything about which points are allowed as endpoints
  - doesn't give a clear meaning of "draw"
  - doesn't say what constitutes a "line" in raster world
  - doesn't say how to measure success of proposed algorithms

Problem Statement

- Given two points P and Q in XY plane, both p and q, with integer coordinates, determine which pixels on raster screen should be on in order to make picture of a unit-width line segment starting at P and ending at Q.

Finding next pixel

Special case:

- Horizontal Line:
  Draw pixel P and increment x coordinate Draw pixel P and increment x coordinate value by 1 to get next pixel.

- Vertical Line:
  Draw pixel P and increment y coordinate value by 1 to get next pixel.

- Diagonal Line:
  Draw pixel P and increment both x and y coordinate by 1 to get next pixel.

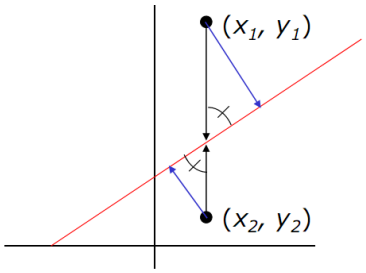
What should we do in general case?

– Increment x coordinate by 1 and choose point closest to line.

– But how do we measure "closest"?

Why can we use vertical distance as measure of which point is closer?

– because vertical distance is proportional to actual distance.
– how do we show this?
– with similar triangles.

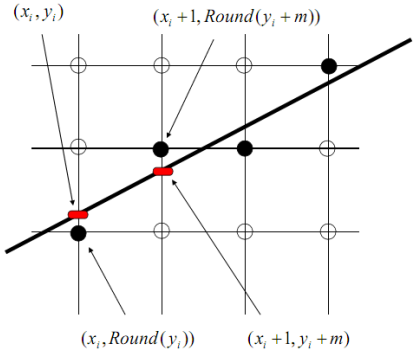


• By similar triangles we can see that true distances to line (in blue) are directly roportional to vertical distances to line (in black) for each point.
• Therefore, point with smaller vertical distance to line is closest to line.

<u>Strategy 1 Incremental Basic Algorithm</u>

• Find equation of line that connects two points P and Q
• Starting with leftmost point P, increment xi by 1 to calculate $y_i = m*x_i + B$
  where m = slope, B = y intercept
• Draw pixel at (xi, Round(y_i)) where Round (y_i) = Floor (0.5 + y_i)

Incremental Algorithm:



- Each iteration requires a floating-point multiplication p
- Modify algorithm to use deltas
  (yi+1 –yi) = m*(xi+1 -xi) + B – B
  yi+1 = yi + m*(xi+1 –xi)

If dx = 1, then yi+1 = yi+ m

- At each step, we make incremental calculations based on preceding step to find next y value.

Example Code

```
// Incremental Line Algorithm
// Assume x0 < x1

void Line(int x0, int y0,
          int x1, int y1) {
   int x, y;
   float      dy = y1 - y0;
   float      dx = x1 - x0;
   float      m = dy / dx;

   y = y0;
   for (x = x0; x < x1; x++) {
       WritePixel(x, Round(y));
       y = y + m;
   }
}
```

Problem with Incremental Algorithm:

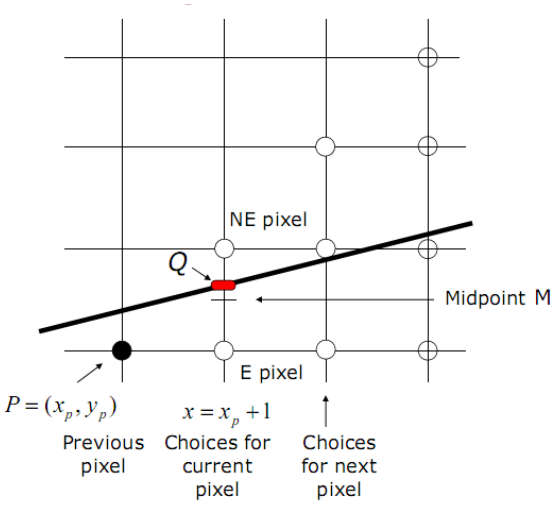
```
void Line(int x0, int y0,
          int x1, int y1) {
   int x, y;
   float      dy = y1 - y0;
   float      dx = x1 - x0;
   float      m = dy / dx;

   y = y0;
   for (x = x0; x < x1; x++) {
       WritePixel(x, Round(y));
       y = y + m;
   }
}
```

Rounding takes time

Since slope is fractional, need special case for vertical lines

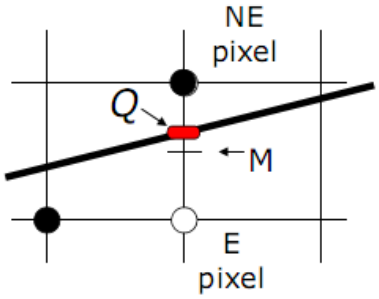
Strategy 2 Midpoint Line Algorithm

- Assume that line's slope is shallow and positive (0 < slope < 1);
- other slopes can be handled by suitable reflections about principle axes reflections about principle axes.
- Call lower left endpoint (x0, y0) and upper right endpoint (x1, y1).
- Assume that we have just selected pixel P at (xp, yp).



- Next, we must choose between pixel to right (E pixel), or one right and one up (NE pixel)
- Let Q be intersection point of line being scan-converted and vertical line x=xp+1.
- Line passes between E and NE.
- Point that is closer to intersection point Q must be chosen.

- Observe on which side of line midpoint M lies:
  - E is closer to line if midpoint M lies above line, i.e., line crosses bottom half
  - NE is closer to line if midpoint M lies below line, i.e., line crosses top half.



- ❖ Error (vertical distance between chosen pixel and actual line) is always <= ½.
  - ❖ Algorithm chooses NE as next pixel for line shown.
- ❖ Now, need to find a way to calculate on which side of line midpoint lies.

Line

Line equation as function *f(x)*:

$$y = mx + B$$

$$y = \frac{dy}{dx}x + B$$

Line equation as implicit function:

$$f(x, y) = ax + by + c = 0$$

for coefficients *a*, *b*, *c*, where *a*, *b* ≠ 0

from above,

$$y \cdot dx = dy \cdot x + B \cdot dx$$

$$dy \cdot x - y \cdot dx + B \cdot dx = 0$$

$$\therefore a = dy, b = -dx, c = B \cdot dx$$

Properties (proof by case analysis):

- $f(x_m, y_m) = 0$ when any point *M* is on line
- $f(x_m, y_m) < 0$ when any point *M* is above line
- $f(x_m, y_m) > 0$ when any point *M* is below line
- Our decision will be based on value of function at midpoint *M* at $(x_p + 1, y_p + ½)$

Decision Variable

Decision Variable *d*:

- We only need sign of $f(x_p + 1, y_p + ½)$ to see where line lies, and then pick nearest pixel
- $d = f(x_p + 1, y_p + ½)$
  - if $d > 0$ choose pixel NE
  - if $d < 0$ choose pixel E
  - if $d = 0$ choose either one consistently

How do we incrementally update *d*?

- On basis of picking E or NE, figure out location of *M* for that pixel, and corresponding value *d* for next grid line
- We can derive *d* for the next pixel based on our current decision

If E was chosen:

Increment M by one in x direction

$$d_{new} = f(x_p + 2, y_p + ½)$$
$$= a(x_p + 2) + b(y_p + ½) + c$$

$$d_{old} = a(x_p + 1) + b(y_p + ½) + c$$

- $d_{new} - d_{old}$ is the incremental difference ΔE

  $d_{new} = d_{old} + a$
  $\Delta E = a = dy$ (2 slides back)

- We can compute value of decision variable at next step incrementally without computing $F(M)$ directly

  $d_{new} = d_{old} + \Delta E = d_{old} + dy$

- ΔE can be thought of as correction or update factor to take d_{old} to d_{new}

- It is referred to as forward difference

If NE was choser

Increment M by one in both x and y di

$$d_{new} = F(x_p + 2, y_p + 3/2)$$
$$= a(x_p + 2) + b(y_p + 3/2)$$

- $\Delta NE = d_{new} - d_{old}$
  $d_{new} = d_{old} + a + b$
  $\Delta NE = a + b = dy - dx$

- Thus, incrementally,
  $d_{new} = d_{old} + \Delta NE = d_{old} + dy - d$

- At each step, algorithm chooses between 2 pixels based on sign of decision variable calculated in previous iteration.

- It then updates decision variable by adding either ΔE or ΔNE to old value depending on choice of pixel. Simple additions only!

- First pixel is first endpoint (x_0, y_0), so we can directly calculate initial value of d for choosing between E and NE.

Example Code

```
void MidpointLine(int x0, int y0,
                  int x1, int y1) {
   int   dx = x1 - x0;
   int   dy = y1 - y0;
   int   d = 2 * dy - dx;
   int   incrE = 2 * dy;
   int   incrNE = 2 * (dy - dx);
   int   x = x0;
   int   y = y0;

   writePixel(x, y);

   while (x < x1) {
        if (d <= 0) {     // East Case
                d = d + incrE;
        } else {          // Northeast Case
                d = d + incrNE;
                y++;
        }
        x++;
        writePixel(x, y);
   }               /* while */
}                  /* MidpointLine */
```

- First midpoint for first $d = d_{start}$ is at $(x_0 + 1, y_0 + ½)$
- $f(x_0 + 1, y_0 + ½)$
  $= a(x_0 + 1) + b(y_0 + ½) + c$
  $= a * x_0 + b * y_0 + c + a + b/2$
  $= f(x_0, y_0) + a + b/2$
- But (x_0, y_0) is point on line and $f(x_0, y_0) = 0$
- Therefore, $d_{start} = a + b/2 = dy - dx/2$
  - use d_{start} to choose second pixel, etc.
- To eliminate fraction in d_{start} :
  - redefine f by multiplying it by 2; $f(x,y) = 2(ax + by + c)$
  - this multiplies each constant and decision variable by 2, but does not change sign
- Bresenham's line algorithm is same but doesn't generalize as nicely to circles and ellipses

CIRCLE-GENERATION ALGORITHMS

Circle is one of the basic graphic component, so in order to understand its generation, let us go through its properties first:

2.5.1 Properties of Circles

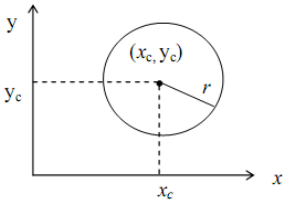


Figure 8: Property of circle

In spite of using the Bresenham circle generation (i.e., incremental approval on basis of decision parameters) we could use basic properties of circle for its generation.

These ways are discussed below:

Generation on the basis of Cartesian Coordinates:

A circle is defined as the set of points or locus of all the points, which exist at a given distance r from center (xc, yc). The distance relationship could be expressed by using Pythagonous theorem in Cartesian coordinates as

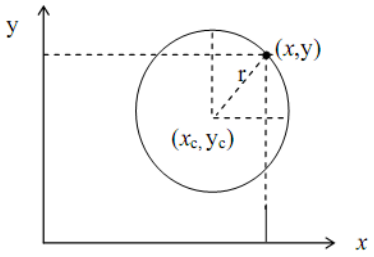


Figure 9: Circle generation (cartesian coordinate system)

$$(x - x_c)^2 + (y - y_c)^2 = r^2 \qquad \text{----------(1)}$$

Now, using (1) we can calculate points on the circumference by stepping along x-axis from $x_c - r$ to $x_c + r$ and calculating respective y values for each position.

$$y = y_c \pm \sqrt{r^2 - (x - x_c)^2} \qquad \text{----------(2)}$$

Generation on basis of polar coordinates (r and θ)

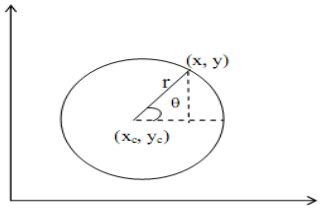


Figure 10: Circle generation (polar coordinate system)

Expressing the circle equation in parametric polar form yields the following equations

$$x = x_c + r\cos\theta$$

$$y = y_c + r\sin\theta \qquad \text{----------(3)}$$

Using a fixed angular step size we can plot a circle with equally spaced points along the circumference.

Note:

- Generation of circle with the help of the two ways mentioned is not a good choice both require a lot of computation equation
  (1) requires square root and multiplication calculations where as
  equation (3) requires trigonometric and multiplication calculations.

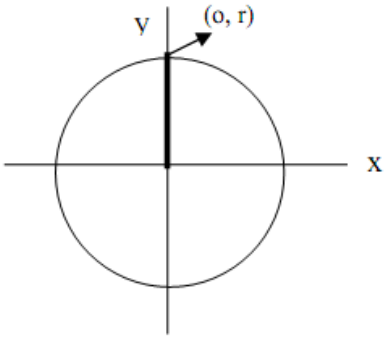
  A more efficient approach is based on incremental calculations of decision parameter. One such approach is Bresenham circle generation. (mid point circle algorithm).

Bresenham circle generation (mid point circle algorithm)

It is similar to line generation. Sample pixels at Unit x intervals and determine the closest pixel to the specified circle path at each step.

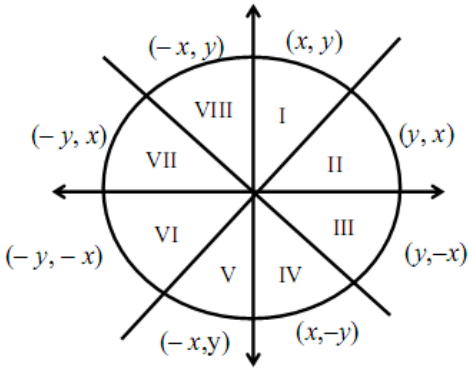
- For a given radius and center position (x, y,) we first setup our algorithm to calculate pixel position around the path of circle centered at coordinate origin (0, 0) i.e., we translate (x_c, y_c).

→ (0, 0) and after the generation we do inverse translation (0, 0) → (x_c, y_c) hence each calculated position (x, y) of circumference is moved to its proper screen position by adding x_c to x and y_c to y.

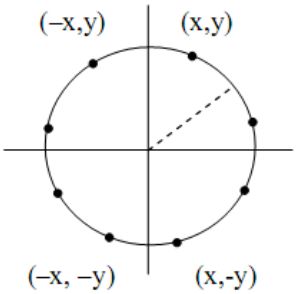


Circle Generation (initialization)

- In Bresenham circle generation (mid point circle algorithm) we calculate points in an octant/quadrant, and then by using symmetry we find other respective points on the circumference.



OCTANT



QUADRANT

Mid Point Circle Generation Algorithm

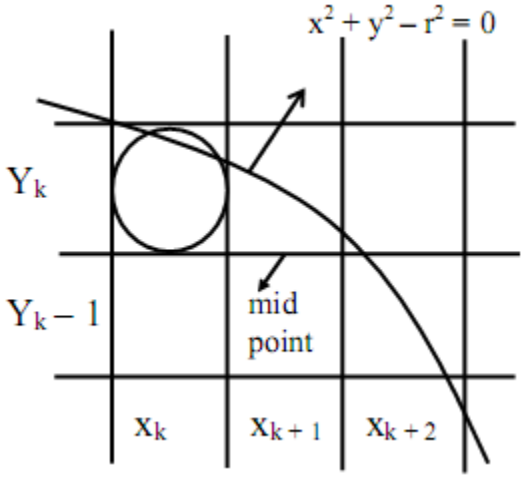
We know that equation of circle is $x^2 + y^2 = r^2$. By rearranging this equation, we can have the function, to be considered for generation of circle as $f_c(x, y)$.

$$f_c(x, y) = x^2 + y^2 - r^2 \qquad \text{----------(1)}$$

The position of any point (x, y) can be analysed by using the sign of $f_c(x, y)$ i.e.,

$$\text{decision parameter} \rightarrow f_c(x, y) = \begin{cases} < 0 \text{ if } (x, y) \text{ is inside circle boundary} \\ = 0 \text{ if } (x, y) \text{ is on the circle boundary} \\ > 0 \text{ if } (x, y) \text{ is outside circle boundary} \end{cases} \qquad \text{---------(2)}$$

i.e., we need to test the sign of $f_c(x, y)$ are performed for mid point between pixels near the circle path at each sampling step.



Mid point between Candidate Pixel at sampling Position x_{k+1} along Circular Path

Figure 13 shows the mid point of two candidate pixel (x_{k+1}, y_k) and (x_{k+1}, y_{k-1}) at sampling position $x_k + 1$.

Assuming we have just plotted the (x_k, y_k) pixel, now, we need to determine the next pixel to be plotted at (x_{k+1}, y_k) or (x_{k+1}, y_{k-1}). In order to decide the pixel on the circles path, we would need to evaluate our decision parameter p_k at mid point between these two pixels i.e.,

$$p_k = f_c\left(x_{k+1}, y_k - \frac{1}{2}\right) = (x_{k+1})^2 + (y_k - \frac{1}{2})^2 - r^2 \qquad \text{----------(3)}$$

(using (1)

If $p_k < 0$ then $\Rightarrow$ midpoint lies inside the circle and pixel on scan line y_k is closer to circle boundary else mid point is outside or on the circle boundary and we select pixel on scan line $y_k - 1$ successive decision parameters are obtained by using incremental calculations.

The recursive way of deciding parameters by evaluating the circle function at sampling positions $x_{k+1} + 1 = (x_k + 1) + 1 = x_k + 2$

$$p_{k+1} = f_c\left(x_{k+1} + 1, y_{k+1} - \frac{1}{2}\right) = [(x_k + 1) + 1)^2 + (y_{k+1} - \frac{1}{2})^2 - r^2] \quad \text{--------(4)}$$

subtract equation (3) from (4) we get

$$p_{k+1} - p_k = \{[(x_k + 1) + 1]^2 + (y_{k+1} - \frac{1}{2})^2 - r^2\} - \{(x_k + 1)^2 + (y_k - \frac{1}{2})^2 - r^2\}$$

$$p_{k+1} - p_k = [(x_k + 1)^2 + 1 + 2(x_k + 1).\ 1] + [y^2_{k+1} + \frac{1}{4} - 2.\frac{1}{2}.\ y_{k+1}] - r^2 - (x_k + 1)^2 - [y_k^2 + \frac{1}{4} - 2.\frac{1}{2}.\ y_k] + r^2$$

$$= (x_k + 1)^2 + 1 + 2(x_k + 1) + y^2_{k+1} + \frac{1}{4} - y_{k+1} - (x_k + 1)^2 - y_k^2 - \frac{1}{4} + y_k$$

$$p_{k+1} = p_k + 2(x_k + 1) + (y^2_{k+1} - y_k^2) - (y_{k+1} - y_k) + 1 \qquad \text{----------(5)}$$

Here, y_{k+1} could be y_k or y_{k-1} depending on sign of p_k

so, increments for obtaining p_{k+1} are : -

either $2x_{k+1} + 1$ (if $p_k < 0$) (*i.e.*, $y_{k+1} = y_k$)

or $2x_{k+1} + 1 - 2y_{k-1}$ (if $p_k > 0$) (*i.e.*, $y_{k+1} = y_k - 1$) *So* ($2y_{k+1} = 2(y_k - 1) = 2y_k - 2$

How do these increments occurs?

- $y_{k+1} = y_k$: use it in (5) we get

  $p_{k+1} = p_k + 2(x_k + 1) + 0 - 0 + 1 = p_k + (2x_{k+1} + 1)$

⇒ $\boxed{\phantom{p_{k+1} = p_k + (2x_{k+1} + 1)}}$ ----------(6)

- $y_{k+1} = y_{k-1}$ use it in (5) we get

  $p_{k+1} = p_k + 2x_{k+1} + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1$

  $= p_k + 2x_{k+1} + \frac{(y_{k+1} - y_k)}{} \frac{[(y_{k+1} + y_k) - 1]}{} + 1$

$\boxed{p_{k+1} = p_k + (2x_{k+1} - 2y_{k-1} + 1)}$ ----------(7)

Also,

$2x_{k+1} = 2(x_k + 1) = 2x_k + 2$ ---------- (8)

$2y_{k+1} \rightarrow 2y_{k-1} = 2(y_k - 1) = 2y_k - 2$ ----------(9)

Note:

- At starting position $(0, r)$ the terms in (8) and (9) have value 0 and $2r$ respectively. Each successive value is obtained by adding 2 to the previous value of $2x$ and subtracting 2 from the previous value of $2y$.

- The initial decision parameter is obtained by evaluating the circle function at start position $(x_0, y_0) = (0, r)$

  i.e., $p_0 = f_c(1, r - \frac{1}{2}) = 1 + (r - \frac{1}{2})^2 - r^2 = \frac{5}{4} - r$ (using 1)

  If radius r is specified as an integer then we can round p_0 to

  $$\boxed{p_0 = 1 - r}$$ (for r as an integer).

Midpoint Circle Algorithm

(a) Input radius r and circle, center (x_c, y_c) and obtain the first point on the circumference of the circle centered on origin as

$$(x_0, y_0) = (0, r)$$

(b) Calculate initial value of decision parameter as

$$p_0 = \frac{5}{4} - r \sim 1 - r$$

(c) At each x_k position starting at $k = 0$ perform following test.
   1) If $p_k < 0$ then next point along the circle centered on (0, 0) is (x_{k+1}, y_k) and $p_{k+1} = p_k + 2x_{k+1} + 1$
   2) Else the next point along circle is $(x_{k+1}, y_k - 1)$ and
      $p_{k+1} = p_k + 2x_{k+1} - 2y_{k+1}$
      where $2x_{k+1} = 2(x_k + 1) = 2x_k + 2$ and $2y_{k-1} = 2(y_k - 1) = 2y_k - 2$

(d) Determine symmetry points in the other seven octants.

(e) Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c) and plot coordinate values $x = x + x_c, y = y + y_c$

(f) Repeat step (c) through (e) until $x \geq y$.

CLIPPING

Clipping may be described as the procedure that identifies the portions of a picture lie inside the region, and therefore, should be drawn or, outside the specified region, and hence, not to be drawn. The algorithms that perform the job of clipping are called clipping algorithms there are various types, such as:

- Point Clipping
- Line Clipping
- Polygon Clipping
- Text Clipping
- Curve Clipping

Further, there are a wide variety of algorithms that are designed to perform certain types of clipping operations, some of them which will be discussed in unit.

Line Clipping Algorithms:
- Cohen Sutherland Line Clippings
- Cyrus-Beck Line Clipping Algorithm

Polygon or Area Clipping Algorithm
- Sutherland-Hodgman Algorithm

There are various other algorithms such as, Liang – Barsky Line clipping, Weiler-Atherton Polygon Clipping, that are quite efficient in performing the task of clipping images. But, we will restrict our discussion to the clipping algorithms mentioned earlier.

Before going into the details of point clipping, let us look at some basic terminologies used in the field of clipping, such as, window and viewport.

Window may be described as the world coordinate area selected for display.

Viewport may be described as the area on a display device on which the window is mapped.

So, it is the window that specifies **what is to be shown or displayed** whereas viewport specifies **where it is to be shown or displayed.**

Specifying these two coordinates, i.e., window and viewport coordinates and then the transformation from window to viewport coordinates is very essential from the point of view of clipping.

Note:
• Assumption: That the window and viewport are rectangular. Then only, by specifying the maximum and the minimum coordinates i.e., (Xw , Yw max max) and (Xw , Yw) we can describe the size of the overall window or viewport. min min
• Window and viewport are not restricted to only rectangular shapes they could be of any other shape (Convex or Concave or both).

For better understanding of the clipping concept refer to *Figure 1*:

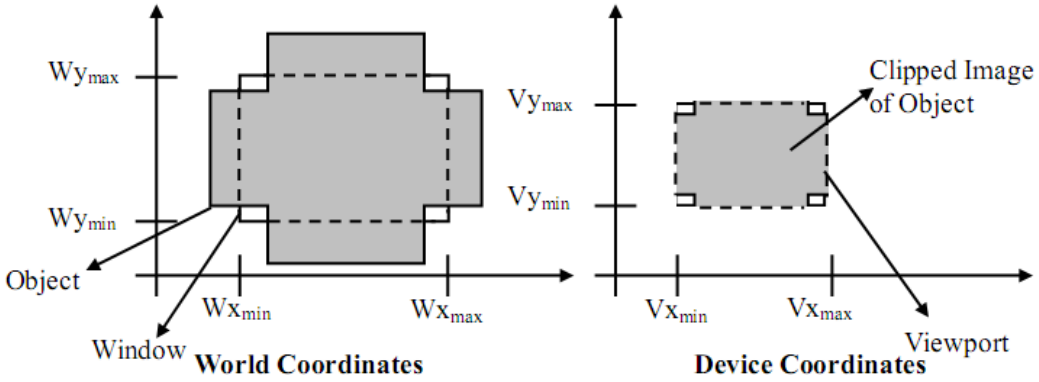


Figure 1: Clipping alongwith Viewing Transformation through rectangular window and viewport

POINT CLIPPING

Point clipping is the technique related to proper display of points in the scene, although, this type of clipping is used less frequently in comparison to other types, i.e., line and polygon clipping. But, in some situations, e.g., the scenes which involve particle movements such as explosion, dust etc., it is quite useful. For the sake of simplicity, let us assume that the clip window is rectangular in shape. So, the minimum and maximum coordinate value, i.e., (Xw_{max}, Yw_{max}) and (Xw_{min}, Yw_{min}) are sufficient to specify window size, and any point (X,Y), which can be shown or displayed should satisfy the following inequalities. Otherwise, the point will not be visible. Thus, the point will be clipped or not can be decided on the basis of following inequalities.

$$Xw_{min} \leq X \leq Xw_{max}$$
$$Yw_{min} \leq Y \leq Yw_{max}$$

It is to be noted that (Xw_{max} , Yw_{max}) and (Xw_{min} , Yw_{min}) can be either world coordinate window boundary or viewport boundary. Further, if any one of these four inequalities is not satisfied, then the point is clipped (not saved for display).

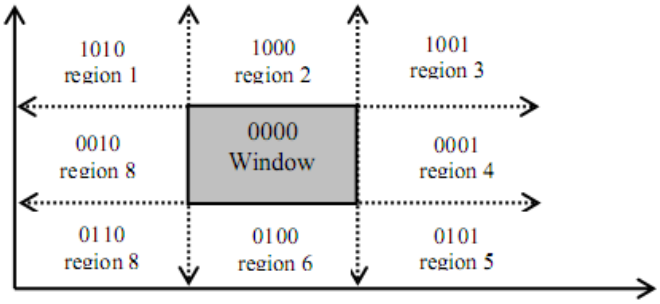
LINE CLIPPING

Line is a series of infinite number of points, where no two points have space in between them. So, the above said inequality also holds for every point on the line to be clipped. A variety of line clipping algorithms are available in the world of computer graphics, but we restrict our discussion to the following Line clipping algorithms, name after their respective developers:

1) Cohen Sutherland algorithm,
2) Cyrus-Beck of algorithm

Cohen Sutherland Line Clippings

This algorithm is quite interesting. The clipping problem is simplified by dividing the area surrounding the window region into four segments Up, Down, Left, Right (U,D,L,R) and assignment of number 1 and 0 to respective segments helps in positioning the region surrounding the window. How this positioning of regions is performed can be well understood by considering Figure.



Positioning of regions surrounding the window

In Figure you might have noticed, that all coding of regions U,D,L,R is done with respect to window region. As window is neither Up nor Down, neither Left nor Right, so, the respective bits UDLR are 0000; now see region 1 of Figure 3. The positioning code UDLR is 1010, i.e., the region 1 lying on the position which is upper left side of the window. Thus, region 1 has UDLR code 1010 (Up so U=1, not Down so D=0, Left so L=1, not Right so R=0).

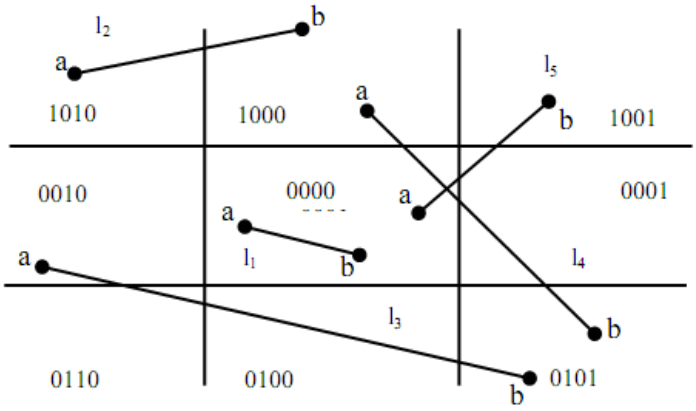
The meaning of the UDLR code to specify the location of region with respect to window is:
1^{st} bit ⇒ Up(U) ; 2^{nd} bit ⇒ Down(D) ;
3^{rd} bit ⇒ Left(L) ; 4^{th} bit ⇒ Right(R),

Now, to perform Line clipping for various line segment which may reside inside the window region fully or partially, or may not even lie in the widow region; we use the tool of logical ANDing between the UDLR codes of the points lying on the line.

Logical ANDing (^) operation => 1 ^ 1 = 1; 1 ^ 0 = 0;
between respective bits implies 0 ^ 1 = 0; 0 ^ 0 = 0

Note:
• UDLR code of window is 0000 always and w.r.t. this will create bit codes of other regions.
• A line segment is visible if both the UDLR codes of the end points of the line segment equal to 0000 i.e. UDLR code of window region. If the resulting code is not 0000 then, that line segment or section of line segment may or may not be visible.

Now, let us study how this clipping algorithm works. For the sake of simplicity we will tackle all the cases with the help of example lines l1 to l5 shown in Figure. Each line segment represents a case.



Various cases of Cohen Sutherland Line Clipping

Note, that in Figure 4, line l_1 is completely visible, l_2 and l_3 are completely invisible; $l_4$and l_5 are partially visible. We will discuss these out comings as three separate cases.

- Case 1: $l_1 \rightarrow$ Completely visible, i.e., Trival acceptance
- (∴both points lie inside the window)

- Case 2: l_2 and $l_3 \rightarrow$ Invisible , i.e., Trival acceptance rejection

- Case 3: l_4 and $l_5 \rightarrow$ partially visible (∴partially inside the window) Now, let us examine these three cases with the help of this algorithm:
    - Case 1: (Trivial acceptance case) if the UDLR bit codes of the end points P,Q of a given line is 0000 then line is completely visible. Here this is the case as the end points a and b of line l1 are: a (0000), b (0000). If this trival acceptance test is failed then, the line segment PQ is passed onto Case 2.

    - Case 2: (Trivial Rejection Case) if the logical intersection (AND) of the bit codes of the end points P, Q of the line segment is ≠ 0000 then line segment is not visible or is rejected.

Note that, in Figure, line 2 is completely on the top of the window but line 3 is neither on top nor at the in bottom plus, either on the LHS nor on the RHS of the window. We use the standard formula of logical ANDing to test the non visibility of the line segment.

So, to test the visibility of line 2 and 3 we need to calculate the logical intersection of end points for line 2 and line 3.

line l_2: bit code of end points are 1010 and 1000 logical intersection of end points = (1010) ^ (1000) = 1000 as logical intersection ≠ 0000. So line 2 will be invisible.

line l_3: end points have bit codes 0010 and 0101 now logical intersection = 0000, i.e., 0010 ^ 0101 = 0000 from the Figure 4, the line is invisible. Similarly in line 4 one end point is on top and the other on the bottom so, logical intersection is 0000 but then it is partially visible, same is true with line 5. These are special cases and we will discuss them in case 3.

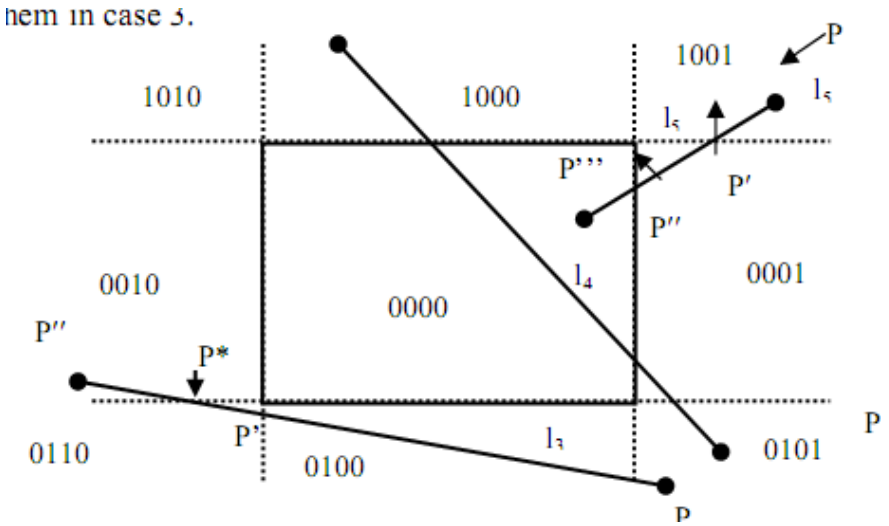


Figure : Cohen Sutherland line clipping

Case 3: Suppose for the line segment PQ, both the trivial acceptance and rejection tests failed (i.e., Case 1 and Case 2 conditions do not hold, this is the case for l_3, l_4 and l_5 line segments) shown above in Figure. For such non-trivial Cases the algorithm is processed, as follows.

Since, both the bitcodes for the end points P, Q of the line segment cannot be equal to 0000. Let us assume that the starting point of the line segment is P whose bit code is not equal to 0000. For example, for the line segment l_5 we choose P to be the bitcodes 1001. Now, scan the bitcode of P from the first bit to the fourth bit and find the position of the bit at which the bit value 1 appears at the first time. For the line segment l_5 it appears at the very first position. If the bit value 1 occurs at the first position then proceed to intersect the line segment with the UP edge of the window and assign the first bit value to its point of intersection as 0. Similarly, if the bit value 1 occurs at the second position while scanning the bit values at the first time then intersect the line segment PQ with the Down edge of the window and so on. This point of intersection may be labeled as P'. Clearly the line segment PP' is outside the window and therefore rejected and the new line segment considered for dipping will be P'Q. The coordinates of P' and its remaining new bit values are computed. Now, by taking P as P', again we have the new line segment PQ which will again be referred to Case 1 for clipping.

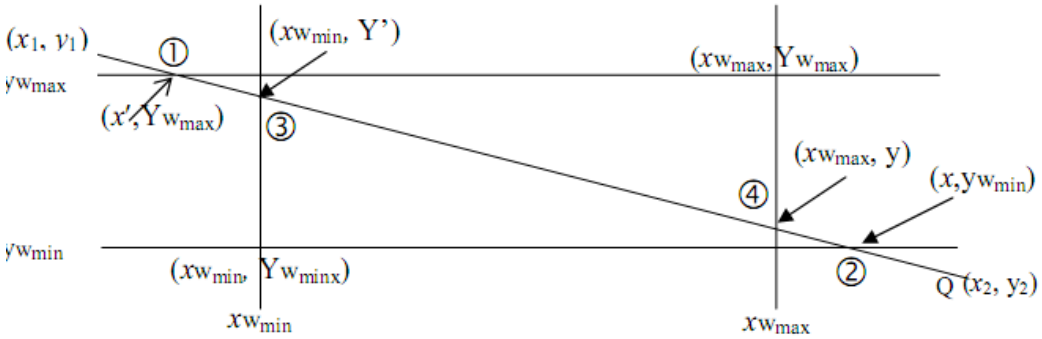


Figure : Line Clipping – Geometrically

Geometrical study of the above type of clipping (it helps to find point of intersection of line PQ with any edge).

Let (x_1, y_1) and (x_2, y_2) be the coordinates of P and Q respectively.

1) top case/above
   if $y_1 > yw_{max}$ then 1st
   bit of bit code = 1 (signifying above) else bit code = 0

2) Bottom case/below case
   if $y1 < yw_{min}$ then 2nd bit = 1 (i.e. below) else bit = 0

3) Left case:
   if $x1 < xw_{min}$ then 3rd bit = 1 (i.e. left) else 0

4) Right case:
   if $x1 > xw_{max}$ then 4th bit = 1 (i.e. right) else 0

Similarly, the bit codes of the point Q will also be assigned.

1) **Top/above case:**

equation of top edge is: $y = yw_{max}$. The equation of line PQ is

$$y - y_1 = m\,(x - x_1)$$

where,

$$m = (y_2 - y_1)/(x_2 - x_1)$$

The coordinates of the point of intersection will be (x, yw_{max}) ∴equation of line between point P and intersection point is

$$(yw_{max} - y_1) = m\,(x - x_1)$$

rearrange we get

$$x = x_1 + \frac{1}{m}(yw_{max} - y_1) \qquad \text{---------- (A)}$$

Hence, we get coordinates (x, yw_{max}) i.e., coordinates of the intersection.

2) **Bottom/below edge** start with $y = yw_{min}$ and proceed as for above case.

∴ equation of line between intersection point (x', yw_{min}) and point Q *i.e.* (x_2, y_2) is

$$(yw_{min} - y_2) = m\,(x' - x_2)$$

rearranging that we get,

$$\boxed{x' = x_2 + \frac{1}{m}(yw_{min} - y_2)} \qquad \text{---------- (B)}$$

The coordinates of the point of intersection of PQ with the bottom edge will be

$$\left(x_2 + \frac{1}{m}(yw_{min} - y_2), yw_{min}\right)$$

3) **Left edge:** the equation of left edge is $x = xw_{min}$.

Now, the point of intersection is (xw_{min}, y).
Using 2 point from the equation of the line we get
$(y - y_1) = m\,(xw_{min} - x_1)$

Rearranging that, we get, $y = y_1 + m\,(xw_{min} - x_1)$. ---------- (C)

Hence, we get value of xw_{min} and y both *i.e.* coordinates of intersection point is given by $(xw_{min}, y_1 + m(xw_{min} - x_1))$.

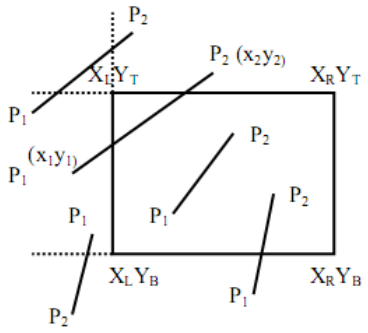
4) **Right edge:** proceed as in left edge case but start with $x\text{-}xw_{max}$.

Now point of intersection is (xw_{max}, y').

Using 2 point form, the equation of the line is
$(y' - y_2) = m\,(xw_{max} - x_2)$

$$y' = y_2 + m\,(xw_{max} - x_2) \quad \text{(D)}$$ ----------

The coordinates of the intersection of PQ with the right edge will be $(xw_{max}, y_2 + m(xw_{max} - x_2))$.



L→Left ;
R→ Right
T→Top ;
B→Bottom

Figure 7: Steps for Cohen Sutherland Line Clipping

STEP 1:

Input: $x_L, x_R, y_T, y_B, P_1(x_1, y_1), P_2(x_2, y_2)$
Initialise $i = 1$
While $i <= 2$
if $x_i < x_L$ then bit 1 of code $-P_i = 1$ else 0
if $x_i > x_R$ then bit 2 of code $-P_i = 1$ else 0 : The endpoint codes of the line are set
if $y_i < y_B$ then bit 3 of code $-P_i = 1$ else 0
if $y_i > y_T$ then bit 4 of code $-P_i = 1$ else 0
i = i +1
end while
i = 1

STEP 2:

Initialise j = 1

While j <= 2

if $x_j < x_L$ then $C_{j\,left} = 1$ else $C_{j\,left} = 0$

if $x_j > x_R$ then $C_{j\,right} = 1$ else $C_{j\,right} = 0$:Set flags according to the position of the line endpoints w.r.t. window edges

if $y_j < y_B$ then $C_{j\,bottom} = 1$ else $C_{j\,bottom} = 0$

if $y_j > y_T$ then $C_{j\,top} = 1$ else $C_{j\,top} = 0$

end while

STEP 3: If codes of P_1 and P_2 are both equal to zero then draw P_1P_2 (totally visible)

STEP 4: If logical intersection or AND operation of code $-P_1$ and code $-P_2$ is not equal to zero then ignore P_1P_2 (totally invisible)

STEP 5: If code $-P_1 = 0$ then swap P_1 and P_2 along with their flags and set i = 1

STEP 6: If code $-P_1 <> 0$ then

for i = 1,

{if $C_{1\,left} = 1$ then

find intersection (x_L, y'_L) with left edge vide eqn. (C)

assign code to (x_L, y'_L)

$P_1 = (x_L, y'_L)$

end if

i = i + 1;

go to 3

}

for i = 2,

{if $C_{1\,right} = 1$ then

find intersection (x_R, y'_R) with right edge vide eqn. (D)

assign code to (x_R, y'_R)

$P_1 = (x_R, y'_R)$

end if

i = i + 1

go to 3

}

```
for i = 3
    {if C1 bottom = 1 then
    find intersection (x'B, yB) with bottom edge vide eqn. (B)
    assign code to (x'B, yB)
    P1 = (x'B, yB)
    end if
    i = i + 1
    go to 3
    }

for i = 4,
    {if C1 top = 1 then
    find intersection (x'T, yT) vide eqn. (A) with top edge
    assign code to (x'T, yT)
    P1 = (x'T, yT)
    end if
    i = i + 1
    go to 3
    }
end
```

Limitation of Cohen Sutherland line Clipping Algorithm

The algorithm is only applicable to rectangular windows and not to any other convex shaped window.

So, a new line-clipping algorithm was developed by Cyrus, and Beck to overcome this limitation. This Cyrus Beck line-clipping Algorithm is capable of clip lining segments irrespective of the shape of the convex window.

i) **Convex shaped windows**: Windows of a shape such that if we take any two points inside the window then the line joining them will never cross the window boundary, such shaped windows are referred as convex shaped windows.

ii) **Concave or non Convex shaped windows**: Windows of a shape such that if we can choose a pair of points inside the window such that the line joining them may cross the window boundary or some section of the line segment may lie outside the window. Such shaped windows are referred to as non-convex or concave shaped windows.

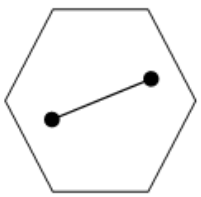


Figure (a)
Convex Polygonal Window

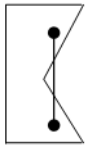


Figure (b)
Non-convex Polygonal window

POLYGON CLIPPING

After understanding the concept of line clipping and its algorithms, we can now extend the concept of line clipping to polygon clipping, because polygon is a surface enclosed by several lines. Thus, by considering the polygon as a set of line we can divide the problem to line clipping and hence, the problem of polygon clipping is simplified. But it is to be noted that, clipping each edge separately by using a line clipping algorithm will certainly not produce a truncated polygon as one would expect. Rather, it would produce a set of unconnected line segments as the polygon is exploded. Herein lies the need to use a different clipping algorithm to output truncated but yet bounded regions from a polygon input. Sutherland-Hodgman algorithm is one of the standard methods used for clipping arbitrary shaped polygons with a rectangular clipping window. It uses divide and conquer technique for clipping the polygon.

Sutherland-Hodgman Algorithm

Any polygon of any arbitrary shape can be described with the help of some set of vertices associated with it. When we try to clip the polygon under consideration with any rectangular window, then, we observe that the coordinates of the polygon vertices satisfies one of the four cases listed in the table shown below, and further it is to be noted that this procedure of clipping can be simplified by clipping the polygon edgewise and not the polygon as a whole. This decomposes the bigger problem into a set of subproblems, which can be handled separately as per the cases listed in the table below. Actually this table describes the cases of the Sutherland-Hodgman Polygon Clipping algorithm.

Thus, in order to clip polygon edges against a window edge we move from vertex Vi to the next vertex Vi+1 and decide the output vertex according to four simple tests or rules or cases listed below:

Table: Cases of the Sutherland-Hodgman Polygon Clipping Algorithm

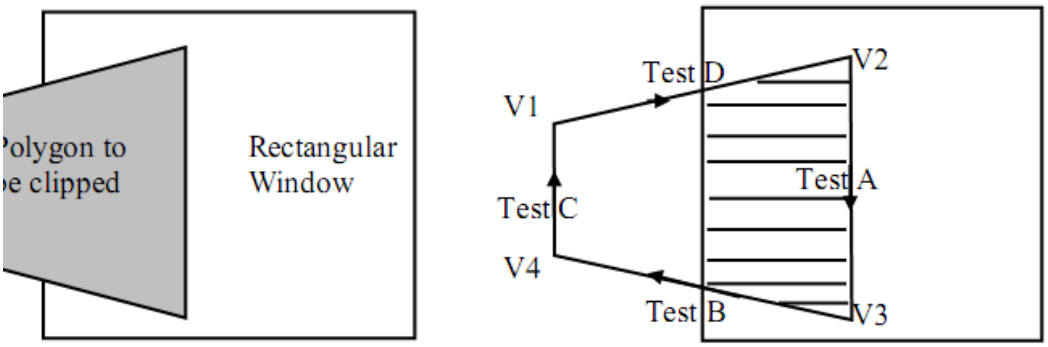
Case	V_i	V_{i+1}	Output Vertex
A	Inside window)	Inside	V_{i+1}
B	Inside	Outside	V'_i i.e intersection of polygon and window edge
C	Outside	Outside	None
D	Outside	Inside	V'_I ; V_{i+1}

In words, the 4 possible Tests listed above to clip any polygon states are as mentioned below:

1) If both Input vertices are inside the window boundary then only 2nd vertex is added to output vertex list.
2) If 1st vertex is inside the window boundary and the 2nd vertex is outside then, only the intersection edge with boundary is added to output vertex.
3) If both Input vertices are outside the window boundary then nothing is added to the output list.
4) If the 1st vertex is outside the window and the 2nd vertex is inside window, then both the intersection points of the polygon edge with window boundary and 2nd vertex are added to output vertex list.
5) So, we can use the rules cited above to clip a polygon correctly. The polygon must be tested against each edge of the clip rectangle; new edges must be added and existing edges must be discarded , retained or divided. Actually this algorithm decomposes the problem of polygon clipping against a clip window into identical subproblems where a subproblem is to clip all polygon edges (pair of vertices) in succession against a single infinite clip edge. The output is a set of clipped edges or pair of vertices that fall in the visible side with respect to clip edge. These set of clipped edges or output vertices are considered as input for the next sub problem of clipping against the second window edge. Thus, considering the output of the previous

subproblem as the input, each of the subproblems are solved sequentially, finally yielding the vertices that fall on or within the window boundary. These vertices connected in order forms, the shape of the clipped polygon.

6) For better understanding of the application of the rules given above consider the Figure 14, where the shaded region shows the clipped polygon.



(a) (b)

Define variables

inVertexArray is the array of input polygon vertices
outVerteArray is the array of output polygon vertices
Nin is the number of entries in inVertexArray
Nout is the number of entries in outVertexArray
n is the number of edges of the clip polygon
ClipEdge[x] is the xth edge of clip polygon defined by a pair of vertices
s, p are the start and end point respectively of current polygon edge
i is the intersection point with a clip boundary
j is the vertex loop counter.

Define Functions

AddNewVertex(newVertex, Nout, outVertexArray) : Adds newVertex to outVertexArray and then updates Nout

InsideTest(testVertex, clipEdge[x]) : Checks whether the vertex lies inside the clip edge or not; returns TRUE is inside
else returns FALSE

Intersect(first, second, clipEdge[x]) //Clip polygon edge(first, second) against clipEdge[x], outputs the intersection point

```
{
    x = 1;
    while (x = n) //     : Loop through all the n clip edges
    {
        Nout = 0 //     : Flush the outVertexArray
        s = inVertexArray[Nin] //: Start with the last vertex in inVertexArray
        for j = 1 to Nin do //    : Loop through Nin number of polygon vertices (edges)
        {
            p = inVertexArrray[j] ;
            if (InsideTest(p, clipEdge[x] = = TRUE))// then    : Case A and D
            {
                if (InsideTest(s, clipEdge[x] = = TRUE))//then
                {
                    AddNewVertex(p, Nout, outVertexArray);  //  :  Case A
                }
```

```
                                        else
                                        {
                                                i = Intersect(s, p, clipEdge[x]);//      : Case D
                                                AddNewVertex(i, Nout, outVertexArray);
                                                AddNewVertex(p, Nout, outVertexArray);
                                        }
                                }
                                else
                                {
                                        if (InsideTest(p, clipEdge[x] == FALSE))//(Cases 2 and 3)
                                        {
                                                if InsideTest(s, clipEdge[x]) = =TRUE)// Case B
                                                {
                                                        Intersect(s, p, clipEdge[x])
                                                        AddNewVertex(i, Nout, outVertexArray)
                                                }
                                        }
                                }

                        //No action for case C
                                s = p   //     : Advance to next pair of vertices
                                j = j + 1
                }

                x = x + 1;
                Nin = Nout;
                inVertexArray = outVertexArray;// : The ouput vertex array for the current clip edge becomes
the input vertexarray for the next clip edge

                }
}
```